Part V

Eigenvalues

Lecture 24. Eigenvalue Problems

Eigenvalue problems are particularly interesting in scientific computing, because the best algorithms for finding eigenvalues are powerful, yet particularly far from obvious. Here, we review the mathematics of eigenvalues and eigenvectors. Algorithms are discussed in later lectures.

Eigenvalues and Eigenvectors

Let $A \in \mathbb{C}^{m \times m}$ be a square matrix. A nonzero vector $x \in \mathbb{C}^m$ is an *eigenvector* of A, and $\lambda \in \mathbb{C}$ is its corresponding *eigenvalue*, if

$$Ax = \lambda x. \tag{24.1}$$

The idea here is that the action of a matrix A on a subspace S of $\mathbb{C}^m$ may sometimes mimic scalar multiplication. When this happens, the special subspace S is called an *eigenspace*, and any nonzero $x \in S$ is an eigenvector.

The set of all the eigenvalues of a matrix A is the *spectrum* of A, a subset of $\mathbb{C}$ denoted by $\Lambda(A)$.

Eigenvalue problems have a very different character from the problems involving square or rectangular linear systems of equations discussed in the previous lectures. For a system of equations, the domain of A could be one space and the range could be a different one. In Example 1.1, for example, A mapped n-vectors of polynomial coefficients to m-vectors of sampled polynomial values. To ask about the eigenvalues of such an A would be meaningless. Eigenvalue problems make sense only when the range and the domain spaces

are the same. This reflects the fact that in applications, eigenvalues are generally used where a matrix is to be compounded iteratively, either explicitly as a power A^k or implicitly in a functional form such as e^{tA}.

Broadly speaking, eigenvalues and eigenvectors are useful for two reasons, one algorithmic, the other physical. Algorithmically, eigenvalue analysis can simplify solutions of certain problems by reducing a coupled system to a collection of scalar problems. Physically, eigenvalue analysis can give insight into the behavior of evolving systems governed by linear equations. The most familiar examples in this latter class are the study of *resonance* (e.g., of musical instruments when struck or plucked or bowed) and of *stability* (e.g., of fluid flows subjected to small perturbations). In such cases eigenvalues tend to be particularly useful for analyzing behavior for large times t. See Exercise 24.3.

Eigenvalue Decomposition

An *eigenvalue decomposition* of a square matrix A, already mentioned in (5.1), is a factorization

$$A = X\Lambda X^{-1}. \tag{24.2}$$

(As we discuss below, such a factorization does not always exist.) Here X is nonsingular and Λ is diagonal.

This definition can be rewritten

$$AX = X\Lambda, \tag{24.3}$$

that is,

$$\begin{bmatrix} & & \\ & A & \\ & & \end{bmatrix} \left[\begin{array}{c|c|c|c} & & & \\ x_1 & x_2 & \cdots & x_m \\ & & & \end{array}\right] = \left[\begin{array}{c|c|c|c} & & & \\ x_1 & x_2 & \cdots & x_m \\ & & & \end{array}\right] \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_m \end{bmatrix}.$$

This makes it clear that if x_j is the jth column of X and λ_j is the jth diagonal entry of Λ, then $Ax_j = \lambda_j x_j$. Thus the jth column of X is an eigenvector of A and the jth entry of Λ is the corresponding eigenvalue.

The eigenvalue decomposition expresses a change of basis to "eigenvector coordinates." If $Ax = b$ and $A = X\Lambda X^{-1}$, we have

$$(X^{-1}b) = \Lambda(X^{-1}x). \tag{24.4}$$

Thus, to compute Ax, we can expand x in the basis of columns of X, apply Λ, and interpret the result as a vector of coefficients of a linear combination of the columns of X.

Geometric Multiplicity

As stated above, the set of eigenvectors corresponding to a single eigenvalue, together with the zero vector, forms a subspace of $\mathbb{C}^m$ known as an *eigenspace*. If λ is an eigenvalue of A, let us denote the corresponding eigenspace by E_λ. An eigenspace E_λ is an example of an *invariant subspace* of A; that is, $AE_\lambda \subseteq E_\lambda$.

The dimension of E_λ can be interpreted as the maximum number of linearly independent eigenvectors that can be found, all with the same eigenvalue λ. This number is known as the *geometric multiplicity* of λ. The geometric multiplicity can also be described as the dimension of the nullspace of $A - \lambda I$, since that nullspace is again E_λ.

Characteristic Polynomial

The *characteristic polynomial* of $A \in \mathbb{C}^{m\times m}$, denoted by p_A or simply p, is the degree m polynomial defined by

$$p_A(z) = \det(zI - A). \tag{24.5}$$

Thanks to the placement of the minus sign, p is *monic:* the coefficient of its degree m term is 1.

Theorem 24.1. *λ is an eigenvalue of A if and only if $p_A(\lambda) = 0$.*

Proof. This follows from the definition of an eigenvalue:

$$\begin{aligned} \lambda \text{ is an eigenvalue} &\iff \text{there is a nonzero vector } x \text{ such that } \lambda x - Ax = 0 \\ &\iff \lambda I - A \text{ is singular} \\ &\iff \det(\lambda I - A) = 0. \end{aligned}$$

□

Theorem 24.1 has an important consequence. *Even if a matrix is real, some of its eigenvalues may be complex.* Physically, this is related to the phenomenon that real dynamical systems can have motions that oscillate as well as grow or decay. Algorithmically, it means that even if the input to a matrix eigenvalue problem is real, the output may have to be complex.

Algebraic Multiplicity

By the fundamental theorem of algebra, we can write p_A in the form

$$p_A(z) = (z - \lambda_1)(z - \lambda_2)\cdots(z - \lambda_m) \tag{24.6}$$

for some numbers $\lambda_j \in \mathbb{C}$. By Theorem 24.1, each λ_j is an eigenvalue of A, and all eigenvalues of A appear somewhere in this list. In general, an eigenvalue might appear more than once. We define the *algebraic multiplicity*

of an eigenvalue λ of A to be its multiplicity as a root of p_A. An eigenvalue is *simple* if its algebraic multiplicity is 1.

The characteristic polynomial gives us an easy way to count the number of eigenvalues of a matrix.

Theorem 24.2. *If $A \in \mathbb{C}^{m\times m}$, then A has m eigenvalues, counted with algebraic multiplicity. In particular, if the roots of p_A are simple, then A has m distinct eigenvalues.*

Note that in particular, every matrix has at least one eigenvalue.

The algebraic multiplicity of an eigenvalue is always at least as great as its geometric multiplicity. To prove this, we need to know something about similarity transformations.

Similarity Transformations

If $X \in \mathbb{C}^{m\times m}$ is nonsingular, then the map $A \mapsto X^{-1}AX$ is called a *similarity transformation* of A. We say that two matrices A and B are *similar* if there is a similarity transformation relating one to the other, i.e., if there exists a nonsingular $X \in \mathbb{C}^{m\times m}$ such that $B = X^{-1}AX$. As described above in the special case of the diagonalization (24.2), any similarity transformation is a change of basis operation.

Many properties are shared by similar matrices A and $X^{-1}AX$.

Theorem 24.3. *If X is nonsingular, then A and $X^{-1}AX$ have the same characteristic polynomial, eigenvalues, and algebraic and geometric multiplicities.*

Proof. The proof that the characteristic polynomials match is a straightforward computation:

$$\begin{aligned} p_{X^{-1}AX}(z) &= \det(zI - X^{-1}AX) = \det(X^{-1}(zI - A)X) \\ &= \det(X^{-1})\det(zI - A)\det(X) = \det(zI - A) = p_A(z). \end{aligned}$$

From the agreement of the characteristic polynomials, the agreement of the eigenvalues and algebraic multiplicities follows. Finally, to prove that the geometric multiplicities agree, we can verify that if E_λ is an eigenspace for A, then $X^{-1}E_\lambda$ is an eigenspace for $X^{-1}AX$, and conversely. □

We can now relate geometric multiplicity to algebraic multiplicity.

Theorem 24.4. *The algebraic multiplicity of an eigenvalue λ is at least as great as its geometric multiplicity.*

Proof. Let n be the geometric multiplicity of λ for the matrix A. Form an $m \times n$ matrix $\hat{V}$ whose n columns constitute an orthonormal basis of the eigenspace $\{x : Ax = \lambda x\}$. Then, extending $\hat{V}$ to a square unitary matrix V, we obtain V^*AV in the form

$$B = V^*AV = \begin{bmatrix} \lambda I & C \\ 0 & D \end{bmatrix}, \tag{24.7}$$

where I is the $n \times n$ identity, C is $n \times (m-n)$, and D is $(m-n) \times (m-n)$. By the definition of the determinant, $\det(zI - B) = \det(zI - \lambda I)\det(zI - D) = (z-\lambda)^n \det(zI - D)$. Therefore the algebraic multiplicity of λ as an eigenvalue of B is at least n. Since similarity transformations preserve multiplicities, the same is true for A. □

Defective Eigenvalues and Matrices

Although a generic matrix has algebraic and geometric multiplicities that are equal (namely, all 1), this is by no means true of every matrix.

Example 24.1. Consider the matrices

$$A = \begin{bmatrix} 2 & & \\ & 2 & \\ & & 2 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 1 & \\ & 2 & 1 \\ & & 2 \end{bmatrix}.$$

Both A and B have characteristic polynomial $(z-2)^3$, so there is a single eigenvalue $\lambda = 2$ of algebraic multiplicity 3. In the case of A, we can choose three independent eigenvectors, for example e_1, e_2, and e_3, so the geometric multiplicity is also 3. For B, on the other hand, we can find only a single independent eigenvector (a scalar multiple of e_1), so the geometric multiplicity of the eigenvalue is only 1. □

An eigenvalue whose algebraic multiplicity exceeds its geometric multiplicity is a *defective eigenvalue*. A matrix that has one or more defective eigenvalues is a *defective matrix*.

Any diagonal matrix is nondefective. For such a matrix, both the algebraic and the geometric multiplicities of an eigenvalue λ are equal to the number of its occurrences along the diagonal.

Diagonalizability

The class of nondefective matrices is precisely the class of matrices that have an eigenvalue decomposition (24.2).

Theorem 24.5. *An $m \times m$ matrix A is nondefective if and only if it has an eigenvalue decomposition $A = X\Lambda X^{-1}$.*

Proof. ($\Longleftarrow$) Given an eigenvalue decomposition $A = X\Lambda X^{-1}$, we know by Theorem 24.3 that Λ is similar to A, with the same eigenvalues and the same multiplicities. Since Λ is a diagonal matrix, it is nondefective, and thus the same holds for A.

($\Longrightarrow$) A nondefective matrix must have m linearly independent eigenvectors, because eigenvectors with different eigenvalues must be linearly independent, and each eigenvalue can contribute as many linearly independent eigenvectors as its multiplicity. If these m independent eigenvectors are formed into the columns of a matrix X, then X is nonsingular and we have $A = X\Lambda X^{-1}$. □

In view of this result, another term for nondefective is *diagonalizable*.

Does a diagonalizable matrix A in some sense "behave like" its diagonal equivalent Λ? The answer depends on what aspect of behavior one measures and on the condition number of X, the matrix of eigenvectors. If X is highly ill-conditioned, then a great deal of information may be discarded in passing from A to Λ. See "A Note of Caution: Nonnormality" in Lecture 34.

Determinant and Trace

The *trace* of $A \in \mathbb{C}^{m\times m}$ is the sum of its diagonal elements: $\mathrm{tr}(A) = \sum_{j=1}^m a_{jj}$. Both the trace and the determinant are related simply to the eigenvalues.

Theorem 24.6. *The determinant* $\det(A)$ *and trace* $\mathrm{tr}(A)$ *are equal to the product and the sum of the eigenvalues of* A*, respectively, counted with algebraic multiplicity:*

$$\det(A) = \prod_{j=1}^m \lambda_j, \qquad \mathrm{tr}(A) = \sum_{j=1}^m \lambda_j. \tag{24.8}$$

Proof. From (24.5) and (24.6), we compute

$$\det(A) \;=\; (-1)^m \det(-A) \;=\; (-1)^m p_A(0) \;=\; \prod_{j=1}^m \lambda_j.$$

This establishes the first formula. As for the second, from (24.5), it follows that the coefficient of the z^{m-1} term of p_A is the negative of the sum of the diagonal elements of A, or $-\mathrm{tr}(A)$. On the other hand, from (24.6), this coefficient is also equal to $-\sum_{j=1}^m \lambda_j$. Thus $\mathrm{tr}(A) = \sum_{j=1}^m \lambda_j$. □

Unitary Diagonalization

It sometimes happens that not only does an $m \times m$ matrix A have m linearly independent eigenvectors, but these can be chosen to be orthogonal. In such

a case, A is *unitarily diagonalizable*, that is, there exists a unitary matrix Q such that

$$A = Q\Lambda Q^*. \tag{24.9}$$

This factorization is both an eigenvalue decomposition and a singular value decomposition, aside from the matter of the signs (possibly complex) of the entries of Λ.

We have already seen a class of matrices that are unitarily diagonalizable: the hermitian matrices. The following result follows from Theorem 24.9, below.

Theorem 24.7. *A hermitian matrix is unitarily diagonalizable, and its eigenvalues are real.*

The hermitian matrices are not the only ones that are unitarily diagonalizable. Other examples include skew-hermitian matrices, unitary matrices, circulant matrices, and any of these plus a multiple of the identity. In general, the class of matrices that are unitarily diagonalizable have an elegant characterization. By definition, we say that a matrix A is *normal* if $A^*A = AA^*$. The following result is well known.

Theorem 24.8. *A matrix is unitarily diagonalizable if and only if it is normal.*

Schur Factorization

One final matrix factorization is actually the one that is most useful in numerical analysis, because all matrices, including defective ones, can be factored in this way. A *Schur factorization* of a matrix A is a factorization

$$A = QTQ^*, \tag{24.10}$$

where Q is unitary and T is upper-triangular. Note that since A and T are similar, the eigenvalues of A necessarily appear on the diagonal of T.

Theorem 24.9. *Every square matrix A has a Schur factorization.*

Proof. We proceed by induction on the dimension m of A. The case $m = 1$ is trivial, so suppose $m \geq 2$. Let x be any eigenvector of A, with corresponding eigenvalue λ. Take x to be normalized and let it be the first column of a unitary matrix U. Then, just as in (24.7), it is easily checked that the product U^*AU has the form

$$U^*AU = \begin{bmatrix} \lambda & B \\ 0 & C \end{bmatrix}.$$

By the inductive hypothesis, there exists a Schur factorization VTV^* of C. Now write

$$Q = U \begin{bmatrix} 1 & 0 \\ 0 & V \end{bmatrix}.$$

This is a unitary matrix, and we have

$$Q^*AQ = \begin{bmatrix} \lambda & BV \\ 0 & T \end{bmatrix}.$$

This is the Schur factorization we seek. □

Eigenvalue-Revealing Factorizations

In the preceding pages we have described three examples of *eigenvalue-revealing factorizations*, factorizations of a matrix that reduce it to a form in which the eigenvalues are explicitly displayed. We can summarize these as follows.

A diagonalization $A = X\Lambda X^{-1}$ exists if and only if A is nondefective.

A unitary diagonalization $A = Q\Lambda Q^*$ exists if and only if A is normal.

A unitary triangularization (Schur factorization) $A = QTQ^*$ always exists.

To compute eigenvalues, we shall construct one of these factorizations. In general, this will be the Schur factorization, since this applies without restriction to all matrices. Moreover, since unitary transformations are involved, the algorithms that result tend to be numerically stable. If A is normal, then the Schur form comes out diagonal, and in particular, if A is hermitian, then we can take advantage of this symmetry throughout the computation and reduce A to diagonal form with half as much work or less than is required for general A.

Exercises

24.1. For each of the following statements, prove that it is true or give an example to show it is false. Throughout, $A \in \mathbb{C}^{m\times m}$ unless otherwise indicated, and "ew" stands for eigenvalue. (This comes from the German "Eigenwert." The corresponding abbreviation for eigenvector is "ev," from "Eigenvektor.")

(a) If λ is an ew of A and $\mu \in \mathbb{C}$, then $\lambda - \mu$ is an ew of $A - \mu I$.

(b) If A is real and λ is an ew of A, then so is $-\lambda$.

(c) If A is real and λ is an ew of A, then so is $\overline{\lambda}$.

(d) If λ is an ew of A and A is nonsingular, then λ^{-1} is an ew of A^{-1}.

(e) If all the ew's of A are zero, then $A = 0$.

(f) If A is hermitian and λ is an ew of A, then $|\lambda|$ is a singular value of A.

(g) If A is diagonalizable and all its ew's are equal, then A is diagonal.

24.2. Here is *Gerschgorin's theorem*, which holds for any $m \times m$ matrix A, symmetric or nonsymmetric. *Every eigenvalue of A lies in at least one of the m circular disks in the complex plane with centers a_{ii} and radii $\sum_{j \neq i} |a_{ij}|$. Moreover, if n of these disks form a connected domain that is disjoint from the other $m - n$ disks, then there are precisely n eigenvalues of A within this domain.*

(a) Prove the first part of Gerschgorin's theorem. (Hint: Let λ be any eigenvalue of A, and x a corresponding eigenvector with largest entry 1.)

(b) Prove the second part. (Hint: Deform A to a diagonal matrix and use the fact that the eigenvalues of a matrix are continuous functions of its entries.)

(c) Give estimates based on Gerschgorin's theorem for the eigenvalues of

$$A = \begin{pmatrix} 8 & 1 & 0 \\ 1 & 4 & \epsilon \\ 0 & \epsilon & 1 \end{pmatrix}, \qquad |\epsilon| < 1.$$

(d) Find a way to establish the tighter bound $|\lambda_3 - 1| \leq \epsilon^2$ on the smallest eigenvalue of A. (Hint: Consider diagonal similarity transformations.)

24.3. Let A be a 10×10 random matrix with entries from the standard normal distribution, minus twice the identity. Write a program to plot $\|e^{tA}\|_2$ against t for $0 \leq t \leq 20$ on a log scale, comparing the result to the straight line $e^{t\alpha(A)}$, where $\alpha(A) = \max_j \mathrm{Re}(\lambda_j)$ is the *spectral abscissa* of A. Run the program for ten random matrices A and comment on the results. What property of a matrix leads to a $\|e^{tA}\|_2$ curve that remains oscillatory as $t \to \infty$?

24.4. For an arbitrary $A \in \mathbb{C}^{m \times m}$ and norm $\|\cdot\|$, prove using Theorem 24.9:

(a) $\lim_{n \to \infty} \|A^n\| = 0 \iff \rho(A) < 1$, where ρ is the spectral radius (Exercise 3.2).

(b) $\lim_{t \to \infty} \|e^{tA}\| = 0 \iff \alpha(A) < 0$, where α is the spectral abscissa.

Lecture 25. Overview of Eigenvalue Algorithms

This and the next five lectures describe some of the classical "direct" algorithms for computing eigenvalues and eigenvectors, as well as a few modern variants. Most of these algorithms proceed in two phases: first, a preliminary reduction from full to structured form; then, an iterative process for the final convergence. This lecture outlines the two-phase approach and explains why it is advantageous.

Shortcomings of Obvious Algorithms

Although eigenvalues and eigenvectors have simple definitions and elegant characterizations, the best ways to compute them are not obvious.

Perhaps the first method one might think of would be to compute the coefficients of the characteristic polynomial and use a rootfinder to extract its roots. Unfortunately, as mentioned in Lecture 15, this strategy is a bad one, because polynomial rootfinding is an ill-conditioned problem in general, even when the underlying eigenvalue problem is well-conditioned. (In fact, polynomial rootfinding is by no means a mainstream topic in scientific computing—precisely because it is so rarely the best way to solve applied problems.)

Another idea would be to take advantage of the fact that the sequence

$$\frac{x}{\|x\|}, \quad \frac{Ax}{\|Ax\|}, \quad \frac{A^2x}{\|A^2x\|}, \quad \frac{A^3x}{\|A^3x\|}, \quad \dots$$

converges, under certain assumptions, to an eigenvector corresponding to the largest eigenvalue of A in absolute value. This method for finding an eigenvector is called *power iteration*. Unfortunately, although power iteration is famous, it is by no means an effective tool for general use. Except for special matrices, it is very slow.

Instead of ideas like these, the best general purpose eigenvalue algorithms are based on a different principle: the computation of an eigenvalue-revealing factorization of A, where the eigenvalues appear as entries of one of the factors. We saw three eigenvalue-revealing factorizations in the last lecture: diagonalization, unitary diagonalization, and unitary triangularization (Schur factorization). In practice, eigenvalues are usually computed by constructing one of these factorizations. Conceptually, what must be done to achieve this is to apply a sequence of transformations to A to introduce zeros in the necessary places, just as in the algorithms we have considered in the preceding lectures of this book. Thus we see that finding eigenvalues ends up rather similar in flavor to solving systems of equations or least squares problems. The algorithms of numerical linear algebra are mainly built upon one technique used over and over again: putting zeros into matrices.

A Fundamental Difficulty

Though the flavors are related, however, a new spice appears in the dish when it comes to computing eigenvalues. What is new is that it would appear that algebraic considerations must preclude the success of any algorithm of this kind.

To see the difficulty, note that just as eigenvalue problems can be reduced to polynomial rootfinding problems, conversely, any polynomial rootfinding problem can be stated as an eigenvalue problem. Suppose we have the monic polynomial

$$p(z) = z^m + a_{m-1}z^{m-1} + \cdots + a_1 z + a_0. \tag{25.1}$$

By expanding in minors, it is not hard to verify that $p(z)$ is equal to $(-1)^m$ times the determinant of the $m \times m$ matrix

$$\begin{bmatrix} -z & & & & & -a_0 \\ 1 & -z & & & & -a_1 \\ & 1 & -z & & & -a_2 \\ & & 1 & \ddots & & \vdots \\ & & & \ddots & -z & -a_{m-2} \\ & & & & 1 & (-z - a_{m-1}) \end{bmatrix}. \tag{25.2}$$

This means that the roots of p are equal to the eigenvalues of the matrix

$$A = \begin{bmatrix} 0 & & & & & -a_0 \\ 1 & 0 & & & & -a_1 \\ & 1 & 0 & & & -a_2 \\ & & 1 & \ddots & & \vdots \\ & & & \ddots & 0 & -a_{m-2} \\ & & & & 1 & -a_{m-1} \end{bmatrix}. \tag{25.3}$$

(We can also get to (25.3) directly, without passing through (25.2), by noting that if z is a root of p, then it follows from (25.1) that $(1, z, z^2, \ldots, z^{m-1})$ is a left eigenvector of A with eigenvalue z.) A is called a *companion matrix* corresponding to p.

Now the difficulty is apparent. It is well known that no formula exists for expressing the roots of an arbitrary polynomial, given its coefficients. This impossibility result is one of the crowning achievements of a body of mathematical work carried out by Abel, Galois, and others in the nineteenth century. Abel proved in 1824 that no analogue of the quadratic formula can exist for polynomials of degree 5 or more.

Theorem 25.1. *For any $m \geq 5$, there is a polynomial $p(z)$ of degree m with rational coefficients that has a real root $p(r) = 0$ with the property that r cannot be written using any expression involving rational numbers, addition, subtraction, multiplication, division, and kth roots.*

This theorem implies that even if we could work in exact arithmetic, there could be no computer program that would produce the exact roots of an arbitrary polynomial in a finite number of steps. It follows that the same conclusion applies to the more general problem of computing eigenvalues of matrices.

This does not mean that we cannot write a good eigenvalue solver. It does mean, however, that such a solver cannot be based on the same kind of techniques that we have used so far for solving linear systems. Methods like Householder reflections and Gaussian elimination would solve linear systems of equations exactly in a finite number of steps if they could be implemented in exact arithmetic. By contrast,

Any eigenvalue solver must be iterative.

The goal of an eigenvalue solver is to produce *sequences of numbers that converge rapidly towards eigenvalues*. In this respect eigenvalue computations are more representative of scientific computing than solutions of linear systems of equations; see the Appendix.

The need to iterate may seem discouraging at first, but the algorithms available in this field converge extraordinarily quickly. In most cases it is

possible to compute sequences of numbers that double or triple the numbers of digits of accuracy at every step. Thus, although computing eigenvalues is an "unsolvable" problem in principle, in practice it differs from the solution of linear systems by only a small constant factor, typically closer to 1 than 10. Theoretically speaking, the dependence of the operation count on $\epsilon_{\text{machine}}$ involves terms as weak as $\log(|\log(\epsilon_{\text{machine}})|)$; see Exercise 25.2.

Schur Factorization and Diagonalization

Most of the general purpose eigenvalue algorithms in use today proceed by computing the Schur factorization. We compute a Schur factorization $A = QTQ^*$ by transforming A by a sequence of elementary unitary similarity transformations $X \mapsto Q_j^* X Q_j$, so that the product

$$\underbrace{Q_j^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_j}_{Q} \tag{25.4}$$

converges to an upper-triangular matrix T as $j \to \infty$.

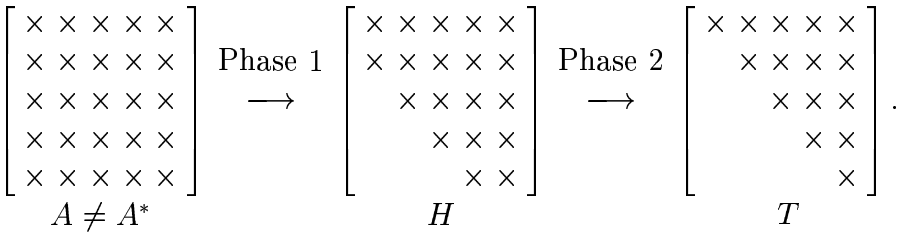
If A is real but not symmetric, then in general it may have complex eigenvalues in conjugate pairs, in which case its Schur form will be complex. Thus an algorithm that computes the Schur factorization will have to be capable of generating complex outputs from real inputs. This can certainly be done; after all, zerofinders for polynomials with real coefficients have the same property. Alternatively, it is possible to carry out the entire computation in real arithmetic if one computes what is known as a *real Schur factorization*. Here, T is permitted to have 2×2 blocks along the diagonal, one for each complex conjugate pair of eigenvalues. This option is important in practice, and is included in all the software libraries, but we shall not give details here.

On the other hand, suppose A is hermitian. Then $Q_j^* \cdots Q_2^* Q_1^* A Q_1 Q_2 \cdots Q_j$ is also hermitian, and thus the limit of the converging sequence is both triangular and hermitian, hence diagonal. This implies that the same algorithms that compute a unitary triangularization of a general matrix also compute a unitary diagonalization of a hermitian matrix. In practice, this is essentially how the hermitian case is typically handled, although various modifications are introduced to take special advantage of the hermitian structure at each step.

Two Phases of Eigenvalue Computations

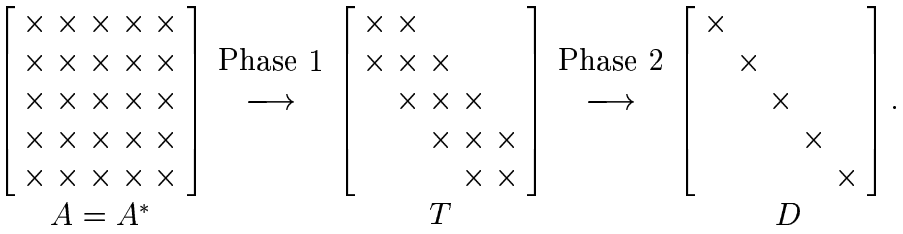
Whether or not A is hermitian, the sequence (25.4) is usually split into two phases. In the first phase, a direct method is applied to produce an *upper-Hessenberg* matrix H, that is, a matrix with zeros below the first subdiagonal. In the second phase, an iteration is applied to generate a formally infinite

sequence of Hessenberg matrices that converge to a triangular form. Schematically, the process looks like this:



The first phase, a direct reduction, requires $O(m^3)$ flops. The second, iterative phase never terminates in principle, and if left to run forever would require an infinite number of flops. However, in practice, convergence to machine precision is achieved in $O(m)$ iterations. Each iteration requires $O(m^2)$ flops, and thus the total work requirement is $O(m^3)$ flops. These figures explain the importance of Phase 1. Without that preliminary step, each iteration of Phase 2 would involve a full matrix, requiring $O(m^3)$ work, and this would bring the total to $O(m^4)$—or higher, since convergence might also sometimes require more than $O(m)$ iterations.

If A is hermitian, the two-phase approach becomes even faster. The intermediate matrix is now a hermitian Hessenberg matrix, that is, *tridiagonal*. The final result is a hermitian triangular matrix, that is, diagonal, as mentioned above. Schematically:



In this hermitian case we shall see that if only eigenvalues are required (not eigenvectors), then each step of Phase 2 can be carried out with only $O(m)$ flops, bringing the total work estimate for Phase 2 to $O(m^2)$ flops. Thus, for hermitian eigenvalue problems, we are in the paradoxical situation that the "infinite" part of the algorithm is in practice not merely as fast as the "finite" part, but an order of magnitude faster.

Exercises

25.1. (a) Let $A \in \mathbb{C}^{m \times m}$ be tridiagonal and hermitian, with all its sub- and superdiagonal entries nonzero. Prove that the eigenvalues of A are distinct. (Hint: Show that for any $\lambda \in \mathbb{C}$, $A - \lambda I$ has rank at least $m - 1$.)

(b) On the other hand, let A be upper-Hessenberg, with all its subdiagonal entries nonzero. Give an example that shows that the eigenvalues of A are not necessarily distinct.

25.2. Let $e_1, e_2, e_3, \ldots$ be a sequence of nonnegative numbers representing errors in some iterative process that converge to zero, and suppose there are a constant C and an exponent α such that for all sufficiently large k, $e_{k+1} \leq C(e_k)^\alpha$. Various algorithms for "Phase 2" of an eigenvalue calculation exhibit *cubic convergence* ($\alpha = 3$), *quadratic convergence* ($\alpha = 2$), or *linear convergence* ($\alpha = 1$ with $C < 1$), which is also, perhaps confusingly, known as *geometric convergence*.

(a) Suppose we want an answer of accuracy $O(\epsilon_{\text{machine}})$. Assuming the amount of work for each step is $O(1)$, show that the total work requirement in the case of linear convergence is $O(\log(\epsilon_{\text{machine}}))$. How does the constant C enter into your work estimate?

(b) Show that in the case of superlinear convergence, i.e., $\alpha > 1$, the work requirement becomes $O(\log(|\log(\epsilon_{\text{machine}})|))$. (Hint: The problem may be simplified by defining a new error measure $f_k = C^{1/(\alpha-1)}e_k$.) How does the exponent α enter into your work estimate?

25.3. Suppose we have a 3×3 matrix and wish to introduce zeros by left- and/or right-multiplications by unitary matrices Q_j such as Householder reflectors or Givens rotations. Consider the following three matrix structures:

$$\text{(a)} \begin{bmatrix} \times & \times & 0 \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix}, \qquad \text{(b)} \begin{bmatrix} \times & \times & 0 \\ \times & 0 & \times \\ 0 & \times & \times \end{bmatrix}, \qquad \text{(c)} \begin{bmatrix} \times & \times & 0 \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix}.$$

For each one, decide which of the following situations holds, and justify your claim.

(i) Can be obtained by a sequence of left-multiplications by matrices Q_j;

(ii) Not (i), but can be obtained by a sequence of left- and right-multiplications by matrices Q_j;

(iii) Cannot be obtained by any sequence of left- and right-multiplications by matrices Q_j.

Lecture 26. Reduction to Hessenberg or Tridiagonal Form

We now describe the first of the two computational phases outlined in the previous lecture: reduction of a full matrix to Hessenberg form by a sequence of unitary similarity transformations. If the original matrix is hermitian, the result is tridiagonal.

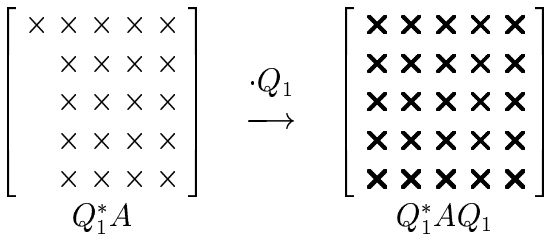
A Bad Idea

To compute the Schur factorization $A = QTQ^*$, we would like to apply unitary similarity transformations to A in such a way as to introduce zeros below the diagonal. A natural first idea might be to attempt direct triangularization by using Householder reflectors to introduce these zeros, one after another.

The first Householder reflector Q_1^*, multiplied on the left of A, would introduce zeros below the diagonal in the first column of A. In the process it will change all of the rows of A. In this and the following diagrams, as usual, entries that are changed at each step are written in boldface:

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}} \quad \overset{Q_1^* \cdot}{\longrightarrow} \quad \underset{Q_1^* A}{\begin{bmatrix} \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \\ \mathbf{0} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} & \mathbf{\times} \end{bmatrix}} .$$

Unfortunately, to complete the similarity transformation, we must also multiply by Q_1 on the right of A:



This has the effect of replacing each column of the matrix by a linear combination of all the columns. The result is that the zeros that were previously introduced are destroyed; we are no better off than when we started.

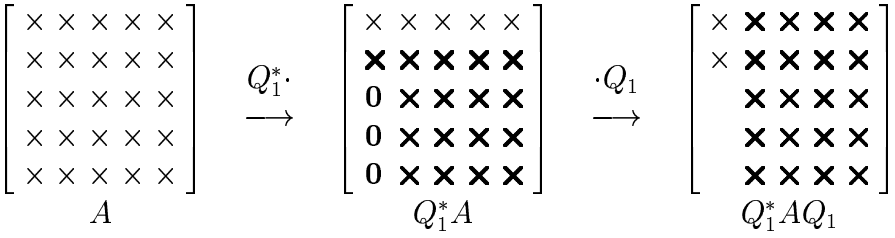
Of course, with hindsight we know that this idea had to fail, because of the "fundamental difficulty" described in the previous lecture. No finite process can reveal the eigenvalues of A exactly.

Curiously, this too-simple strategy, which appears futile as we have discussed it, does have the effect, typically, of reducing the size of the entries below the diagonal, even if it does not make them zero. We shall return to this "bad idea" when we discuss the QR algorithm.

A Good Idea

The right strategy for introducing zeros in Phase 1 is to be less ambitious and operate on fewer entries of the matrix. We shall only conquer territory we are sure we can defend.

At the first step, we select a Householder reflector Q_1^* that leaves the first row unchanged. When it is multiplied on the left of A, it forms linear combinations of only rows $2, \ldots, m$ to introduce zeros into rows $3, \ldots, m$ of the first column. Then, when Q_1 is multiplied on the right of $Q_1^* A$, it leaves the first column unchanged. It forms linear combinations of columns $2, \ldots, m$ and does not alter the zeros that have been introduced:



This idea is repeated to introduce zeros into subsequent columns. For example, the second Householder reflector, Q_2, leaves the first and second rows and

columns unchanged:

$$\underset{Q_1^*AQ_1}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & \times & \times & \times & \times \end{bmatrix}} \xrightarrow{Q_2^*\cdot} \underset{Q_2^*Q_1^*AQ_1}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & 0 & \times & \times & \times \\ & 0 & \times & \times & \times \end{bmatrix}} \xrightarrow{\cdot Q_2} \underset{Q_2^*Q_1^*AQ_1Q_2}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}}.$$

After repeating this process $m-2$ times, we have a product in Hessenberg form, as desired:

$$\underbrace{Q_{m-2}^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_{m-2}}_{Q} = \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix} = H.$$

The algorithm is formulated below; compare Algorithm 10.1.

Algorithm 26.1. Householder Reduction to Hessenberg Form

for $k = 1$ **to** $m-2$

$x = A_{k+1:m,k}$

$v_k = \operatorname{sign}(x_1)\|x\|_2 e_1 + x$

$v_k = v_k / \|v_k\|_2$

$A_{k+1:m,k:m} = A_{k+1:m,k:m} - 2v_k(v_k^* A_{k+1:m,k:m})$

$A_{1:m,k+1:m} = A_{1:m,k+1:m} - 2(A_{1:m,k+1:m} v_k)v_k^*$

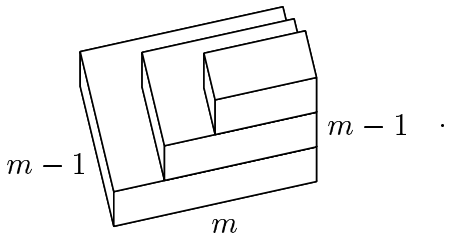
Just as in Algorithm 10.1, here the matrix $Q = \prod_{k=1}^{m-2} Q_k$ is never formed explicitly. The reflection vectors v_k are saved instead, and can be used to multiply by Q or reconstruct Q later if necessary. For details, see Lecture 10.

Operation Count

The number of operations required by Algorithm 26.1 can be counted with the same geometric reasoning we have used before. The rule of thumb is that unitary operations require four flops for each element operated upon.

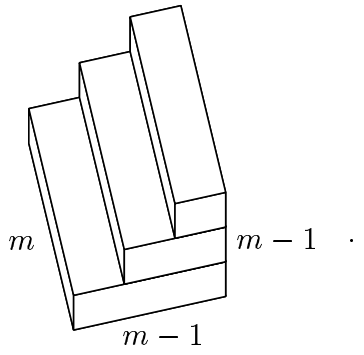
The work is dominated by the two updates of submatrices of A. The first loop applies a Householder reflector on the left of the matrix. The kth such reflector operates on the last $m-k$ rows. Since at the time the reflector is applied, these rows have zeros in the first $k-1$ columns, arithmetic has to be

performed only on the last $m-k+1$ entries of each row. The picture is as follows:



As $m \to \infty$, the volume converges to $\frac{1}{3}m^3$. At four flops per element, the amount of work in this loop is $\sim \frac{4}{3}m^3$ flops.

The second inner loop applies a Householder reflector on the right of the matrix. At the kth step, the reflector operates by forming linear combinations of the last $m-k$ columns. This loop involves more work than the first one because there are no zeros that can be ignored. Arithmetic must be performed on all of the m entries of each of the columns operated upon, a total of $m(m-k)$ entries for a single value of k. The picture looks like this:



The volume converges as $m \to \infty$ to $\frac{1}{2}m^3$, so, at four flops per element, this second loop requires $\sim 2m^3$ flops.

All together, the total amount of work for unitary reduction of an $m \times m$ matrix to Hessenberg form is:

$$\text{Work for Hessenberg reduction:} \quad \sim \frac{10}{3}m^3 \text{ flops.} \tag{26.1}$$

The Hermitian Case: Reduction to Tridiagonal Form

If A is hermitian, the algorithm just described will reduce A to tridiagonal form (at least, in the absence of rounding errors). This is easy to see: since A is hermitian, Q^*AQ is also hermitian, and any hermitian Hessenberg matrix is tridiagonal.

Since zeros are now introduced in rows as well as columns, additional arithmetic can be avoided by ignoring these additional zeros. With this optimization, applying a Householder reflector on the right is as cheap as applying

the reflector on the left, and the total cost of applying the right reflectors is reduced from $2m^3$ to $\frac{4}{3}m^3$ flops. We have two pyramids to add up instead of a pyramid and a prism, and the total amount of arithmetic is reduced to $\frac{8}{3}m^3$ flops.

This saving, however, is based only on sparsity, not symmetry. In fact, at every stage of the computation, the matrix being operated upon is hermitian. This gives another factor of two that can be taken advantage of, bringing the total work estimate to

$$\text{Work for tridiagonal reduction:} \quad \sim \frac{4}{3}m^3 \text{ flops.} \tag{26.2}$$

We shall not give the details of the implementation.

Stability

Like the Householder algorithm for QR factorization, the algorithm just described is backward stable. Recall from Theorem 16.1 that, for any $A \in \mathbb{C}^{m\times n}$, the Householder algorithm for QR factorization computes reflection vectors equivalent to an implicit, exactly unitary factor $\tilde{Q}$ (16.2), as well as an explicit upper-triangular factor $\tilde{R}$, such that

$$\tilde{Q}\tilde{R} = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}).$$

The same kind of error estimate can be established for Algorithm 26.1. Let $\tilde{H}$ be the actual Hessenberg matrix computed in floating point arithmetic, and let $\tilde{Q}$, as before, be the exactly unitary matrix (16.2) corresponding to the reflection vectors $\tilde{v}_k$ computed in floating point arithmetic. The following result can be proved.

Theorem 26.1. *Let the Hessenberg reduction $A = QHQ^*$ of a matrix $A \in \mathbb{C}^{m\times m}$ be computed by Algorithm* 26.1 *on a computer satisfying the axioms* (13.5) *and* (13.7)*, and let the computed factors $\tilde{Q}$ and $\tilde{H}$ be defined as indicated above. Then we have*

$$\tilde{Q}\tilde{H}\tilde{Q}^* = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{26.3}$$

for some $\delta A \in \mathbb{C}^{m\times m}$.

Exercises

26.1. Theorem 26.1 and its successors in later lectures show that we can compute eigenvalues $\{\tilde{\lambda}_k\}$ of A numerically that are the exact eigenvalues of a

matrix $A+\delta A$ with $\|\delta A\|/\|A\| = O(\epsilon_{\text{machine}})$. Does this mean they are close to the exact eigenvalues $\{\lambda_k\}$ of A? This is a question of eigenvalue perturbation theory.

One can approach such problems geometrically as follows. Given $A \in \mathbb{C}^{m\times m}$ with spectrum $\Lambda(A) \subseteq \mathbb{C}$ and $\epsilon > 0$, define the 2-norm *ϵ-pseudospectrum of A*, $\Lambda_\epsilon(A)$, to be the set of numbers $z \in \mathbb{C}$ satisfying any of the following conditions:

(i) z is an eigenvalue of $A + \delta A$ for some δA with $\|\delta A\|_2 \le \epsilon$;

(ii) There exists a vector $u \in \mathbb{C}^m$ with $\|(A - zI)u\|_2 \le \epsilon$ and $\|u\|_2 = 1$;

(iii) $\sigma_m(zI - A) \le \epsilon$;

(iv) $\|(zI - A)^{-1}\|_2 \ge \epsilon^{-1}$.

The matrix $(zI - A)^{-1}$ in (iv) is known as the *resolvent* of A at z; if z is an eigenvalue of A, we use the convention $\|(zI - A)^{-1}\|_2 = \infty$. In (iii), σ_m denotes the smallest singular value.

Prove that conditions (i)–(iv) are equivalent.

26.2. Let A be the 32×32 matrix with -1 on the main diagonal, 1 on the first and second superdiagonals, and 0 elsewhere.

(a) Using an SVD algorithm built into MATLAB or another software system, together with contour plotting software, generate a plot of the boundaries of the 2-norm ϵ-pseudospectra of A for $\epsilon = 10^{-1}, 10^{-2}, \ldots, 10^{-8}$.

(b) Produce a semilogy plot of $\|e^{tA}\|_2$ against t for $0 \le t \le 50$. What is the initial growth rate of the curve before the eventual decay sets in? Can you relate this to your plot of pseudospectra? (Compare Exercise 24.3.)

26.3. One of the best known results of eigenvalue perturbation theory is the *Bauer–Fike theorem. Suppose $A \in \mathbb{C}^{m\times m}$ is diagonalizable with $A = V\Lambda V^{-1}$, and let $\delta A \in \mathbb{C}^{m\times m}$ be arbitrary. Then every eigenvalue of $A + \delta A$ lies in at least one of the m circular disks in the complex plane of radius $\kappa(V)\|\delta A\|_2$ centered at the eigenvalues of A, where κ is the 2-norm condition number.* (Compare Exercise 24.2.)

(a) Prove the Bauer–Fike theorem by using the equivalence of conditions (i) and (iv) of Exercise 26.1.

(b) Suppose A is normal. Show that for each eigenvalue $\tilde{\lambda}_j$ of $A + \delta A$, there is an eigenvalue λ_j of A such that

$$|\tilde{\lambda}_j - \lambda_j| \;\le\; \|\delta A\|_2. \tag{26.4}$$

Lecture 27. Rayleigh Quotient, Inverse Iteration

In this lecture we present some classical eigenvalue algorithms. Individually, these tools are useful in certain circumstances—especially inverse iteration, which is the standard method for determining an eigenvector when the corresponding eigenvalue is known. Combined, they are the ingredients of the celebrated QR algorithm, described in the next two lectures.

Restriction to Real Symmetric Matrices

Throughout numerical linear algebra, most algorithmic ideas are applicable either to general matrices or, with certain simplifications, to hermitian matrices. For the topics discussed in this and the next three lectures, this continues to be at least partly true, but some of the differences between the general and the hermitian cases are rather sizable. Therefore, in these four lectures, we simplify matters by considering only matrices that are real and symmetric. We also assume throughout that $\|\cdot\| = \|\cdot\|_2$.

Thus, for these four lectures: $A = A^T \in \mathbb{R}^{m\times m}$, $x \in \mathbb{R}^m$, $x^* = x^T$, $\|x\| = \sqrt{x^T x}$. In particular, this means that A has real eigenvalues and a complete set of orthogonal eigenvectors. We use the following notation:

real eigenvalues: $\lambda_1, \ldots, \lambda_m$,

orthonormal eigenvectors: $q_1, \ldots, q_m$.

The eigenvectors are presumed normalized by $\|q_j\| = 1$, and the ordering of the eigenvalues will be specified as necessary.

Most of the ideas to be described in the next few lectures pertain to Phase 2 of the two phases described in Lecture 25. This means that by the time we come to applying these ideas, A will be not just real and symmetric, but tridiagonal. This tridiagonal structure is occasionally of mathematical importance, for example in choosing shifts for the QR algorithm, and it is always of algorithmic importance, reducing many steps from $O(m^3)$ to $O(m)$ flops, as discussed at the end of the lecture.

Rayleigh Quotient

The *Rayleigh quotient* of a vector $x \in \mathbb{R}^m$ is the scalar

$$r(x) = \frac{x^T A x}{x^T x}. \tag{27.1}$$

Notice that if x is an eigenvector, then $r(x) = \lambda$ is the corresponding eigenvalue. One way to motivate this formula is to ask: given x, what scalar α "acts most like an eigenvalue" for x in the sense of minimizing $\|Ax - \alpha x\|_2$? This is an $m \times 1$ least squares problem of the form $x\alpha \approx Ax$ (x is the matrix, α is the unknown vector, Ax is the right-hand side). By writing the normal equations (11.9) for this system, we obtain the answer: $\alpha = r(x)$. Thus $r(x)$ is a natural eigenvalue estimate to consider if x is close to, but not necessarily equal to, an eigenvector.

To make these ideas quantitative, it is fruitful to view $x \in \mathbb{R}^m$ as a variable, so that r is a function $\mathbb{R}^m \to \mathbb{R}$. We are interested in the local behavior of $r(x)$ when x is near an eigenvector. One way to approach this question is to calculate the partial derivatives of $r(x)$ with respect to the coordinates x_j:

$$\begin{aligned} \frac{\partial r(x)}{\partial x_j} &= \frac{\frac{\partial}{\partial x_j}(x^T A x)}{x^T x} - \frac{(x^T A x)\frac{\partial}{\partial x_j}(x^T x)}{(x^T x)^2} \\ &= \frac{2(Ax)_j}{x^T x} - \frac{(x^T A x)2x_j}{(x^T x)^2} \quad = \quad \frac{2}{x^T x}\left(Ax - r(x)x\right)_j . \end{aligned}$$

If we collect these partial derivatives into an m-vector, we find we have calculated the *gradient* of $r(x)$, denoted by $\nabla r(x)$. We have shown:

$$\nabla r(x) = \frac{2}{x^T x}(Ax - r(x)x). \tag{27.2}$$

From this formula we see that at an eigenvector x of A, the gradient of $r(x)$ is the zero vector. Conversely, if $\nabla r(x) = 0$ with $x \neq 0$, then x is an eigenvector and $r(x)$ is the corresponding eigenvalue.

Geometrically speaking, the eigenvectors of A are the *stationary points* of the function $r(x)$, and the eigenvalues of A are the values of $r(x)$ at these

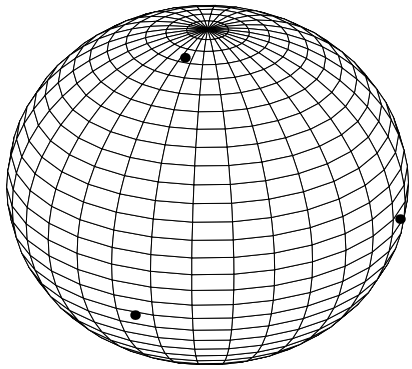


Figure 27.1. *The Rayleigh quotient $r(x)$ is a continuous function on the unit sphere $\|x\| = 1$ in $\mathbb{R}^m$, and the stationary points of $r(x)$ are the normalized eigenvectors of A. In this example with $m = 3$, there are three orthogonal stationary points (as well as their antipodes).*

stationary points. Actually, since $r(x)$ is independent of the scale of x, these stationary points lie along lines through the origin in $\mathbb{R}^m$. If we normalize by restricting attention to the unit sphere $\|x\| = 1$, they become isolated points (assuming that the eigenvalues of A are simple), as suggested in Figure 27.1.

Let q_J be one of the eigenvectors of A. From the fact that $\nabla r(q_J) = 0$, together with the smoothness of the function $r(x)$ (everywhere except at the origin $x = 0$), we derive an important consequence:

$$r(x) - r(q_J) = O(\|x - q_J\|^2) \quad \text{as } x \to q_J. \tag{27.3}$$

Thus the Rayleigh quotient is a *quadratically accurate* estimate of an eigenvalue. Herein lies its power.

A more explicit way to derive (27.3) is to expand x as a linear combination of the eigenvectors $q_1, \ldots, q_m$ of A. If $x = \sum_{j=1}^m a_j q_j$, then $r(x) = \sum_{j=1}^m a_j^2 \lambda_j / \sum_{j=1}^m a_j^2$. Thus $r(x)$ is a weighted mean of the eigenvalues of A, with the weights equal to the squares of the coordinates of x in the eigenvector basis. Because of this squaring of the coordinates, it is not hard to see that if $|a_j/a_J| \le \epsilon$ for all $j \ne J$, then $r(x) - r(q_J) = O(\epsilon^2)$.

Power Iteration

Now we switch tacks. Suppose $v^{(0)}$ is a vector with $\|v^{(0)}\| = 1$. The following process, *power iteration*, was cited as a not especially good idea at the beginning of Lecture 25. It may be expected to produce a sequence $v^{(i)}$ that converges to an eigenvector corresponding to the largest eigenvalue of A.

Algorithm 27.1. Power Iteration

$v^{(0)}$ = some vector with $\|v^{(0)}\| = 1$
for $k = 1, 2, \ldots$
  $w = Av^{(k-1)}$ — apply A
  $v^{(k)} = w/\|w\|$ — normalize
  $\lambda^{(k)} = (v^{(k)})^T A v^{(k)}$ — Rayleigh quotient

In this and the algorithms to follow, we give no attention to termination conditions, describing the loop only by the suggestive expression "**for** $k = 1, 2, \ldots$." Of course, in practice, termination conditions are very important, and this is one of the points where top-quality software such as can be found in LAPACK or MATLAB is likely to be superior to a program an individual might write.

We can analyze power iteration easily. Write $v^{(0)}$ as a linear combination of the orthonormal eigenvectors q_i:

$$v^{(0)} = a_1 q_1 + a_2 q_2 + \cdots + a_m q_m.$$

Since $v^{(k)}$ is a multiple of $A^k v^{(0)}$, we have for some constants c_k

$$\begin{aligned} v^{(k)} &= c_k A^k v^{(0)} \\ &= c_k (a_1 \lambda_1^k q_1 + a_2 \lambda_2^k q_2 + \cdots + a_m \lambda_m^k q_m) \\ &= c_k \lambda_1^k \left(a_1 q_1 + a_2 (\lambda_2/\lambda_1)^k q_2 + \cdots + a_m (\lambda_m/\lambda_1)^k q_m \right). \end{aligned} \tag{27.4}$$

From here we obtain the following conclusion.

Theorem 27.1. *Suppose* $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m| \geq 0$ *and* $q_1^T v^{(0)} \neq 0$. *Then the iterates of Algorithm* 27.1 *satisfy*

$$\|v^{(k)} - (\pm q_1)\| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right), \qquad |\lambda^{(k)} - \lambda_1| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \right) \tag{27.5}$$

as $k \to \infty$. *The* $\pm$ *sign means that at each step* k, *one or the other choice of sign is to be taken, and then the indicated bound holds.*

Proof. The first equation follows from (27.4), since $a_1 = q_1^T v^{(0)} \neq 0$ by assumption. The second follows from this and (27.3). If $\lambda_1 > 0$, then the $\pm$ signs are all $+$ or all $-$, whereas if $\lambda_1 < 0$, they alternate. □

The $\pm$ signs in (27.5) and in similar equations below are not very appealing. There is an elegant way to avoid these complications, which is to speak of convergence of subpaces, not vectors—to say that $\langle v^{(k)} \rangle$ converges to $\langle q_1 \rangle$, for

example. However, we shall not do this, in order to avoid getting into the details of how convergence of subspaces can be made precise.

On its own, power iteration is of limited use, for several reasons. First, it can find only the eigenvector corresponding to the largest eigenvalue. Second, the convergence is linear, reducing the error only by a constant factor $\approx |\lambda_2/\lambda_1|$ at each iteration. Finally, the quality of this factor depends on having a largest eigenvalue that is significantly larger than the others. If the largest two eigenvalues are close in magnitude, the convergence will be very slow.

Fortunately, there is a way to amplify the differences between eigenvalues.

Inverse Iteration

For any $\mu \in \mathbb{R}$ that is not an eigenvalue of A, the eigenvectors of $(A - \mu I)^{-1}$ are the same as the eigenvectors of A, and the corresponding eigenvalues are $\{(\lambda_j - \mu)^{-1}\}$, where $\{\lambda_j\}$ are the eigenvalues of A. This suggests an idea. Suppose μ is close to an eigenvalue λ_J of A. Then $(\lambda_J - \mu)^{-1}$ may be much larger than $(\lambda_j - \mu)^{-1}$ for all $j \neq J$. Thus, if we apply power iteration to $(A - \mu I)^{-1}$, the process will converge rapidly to q_J. This idea is called *inverse iteration*.

Algorithm 27.2. Inverse Iteration

$v^{(0)}$ = some vector with $\|v^{(0)}\| = 1$
for $k = 1, 2, \ldots$

Solve $(A - \mu I)w = v^{(k-1)}$ for w	apply $(A - \mu I)^{-1}$
$v^{(k)} = w/\|w\|$	normalize
$\lambda^{(k)} = (v^{(k)})^T A v^{(k)}$	Rayleigh quotient

What if μ is an eigenvalue of A, so that $A - \mu I$ is singular? What if it is nearly an eigenvalue, so that $A - \mu I$ is so ill-conditioned that an accurate solution of $(A - \mu I)w = v^{(k-1)}$ cannot be expected? These apparent pitfalls of inverse iteration cause no trouble at all; see Exercise 27.5.

Like power iteration, inverse iteration exhibits only linear convergence. Unlike power iteration, however, we can choose the eigenvector that will be found by supplying an estimate μ of the corresponding eigenvalue. Furthermore, the rate of linear convergence can be controlled, for it depends on the quality of μ. If μ is much closer to one eigenvalue of A than to the others, then the largest eigenvalue of $(A - \mu I)^{-1}$ will be much larger than the rest. Using the same reasoning as with power iteration, we obtain the following theorem.

Theorem 27.2. *Suppose λ_J is the closest eigenvalue to μ and λ_K is the second closest, that is, $|\mu - \lambda_J| < |\mu - \lambda_K| \leq |\mu - \lambda_j|$ for each $j \neq J$. Furthermore,*

suppose $q_J^T v^{(0)} \neq 0$. *Then the iterates of Algorithm* 27.2 *satisfy*

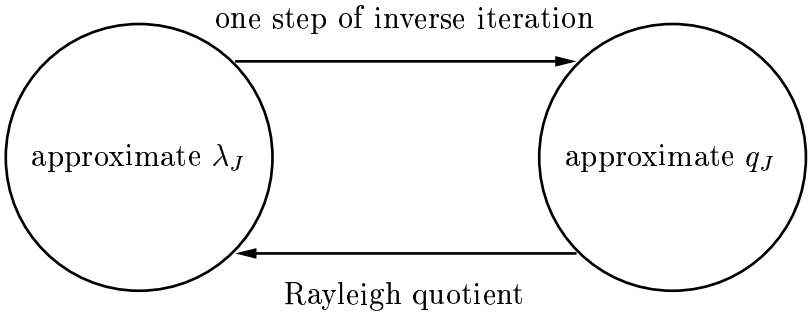
$$\|v^{(k)} - (\pm q_J)\| = O\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^k\right), \qquad |\lambda^{(k)} - \lambda_J| = O\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^{2k}\right)$$

as $k \to \infty$, *where the* $\pm$ *sign has the same meaning as in Theorem* 27.1.

Inverse iteration is one of the most valuable tools of numerical linear algebra, for it is the standard method of calculating one or more eigenvectors of a matrix if the eigenvalues are already known. In this case Algorithm 27.2 is applied as written, except that the calculation of the Rayleigh quotient is dispensed with.

Rayleigh Quotient Iteration

So far in this lecture, we have presented one method for obtaining an eigenvalue estimate from an eigenvector estimate (the Rayleigh quotient), and another method for obtaining an eigenvector estimate from an eigenvalue estimate (inverse iteration). The possibility of combining these ideas is irresistible:



(The figure is oversimplified; to get from an approximate λ_J to an approximate q_J by a step of inverse iteration, one also needs a preliminary approximation to q_J.) The idea is to use continually improving eigenvalue estimates to increase the rate of convergence of inverse iteration at every step. This algorithm is called *Rayleigh quotient iteration*.

Algorithm 27.3. Rayleigh Quotient Iteration

$v^{(0)}$ = some vector with $\|v^{(0)}\| = 1$
$\lambda^{(0)} = (v^{(0)})^T A v^{(0)}$ = corresponding Rayleigh quotient
for $k = 1, 2, \ldots$

Solve $(A - \lambda^{(k-1)} I) w = v^{(k-1)}$ for w	apply $(A - \lambda^{(k-1)} I)^{-1}$
$v^{(k)} = w / \|w\|$	normalize
$\lambda^{(k)} = (v^{(k)})^T A v^{(k)}$	Rayleigh quotient

The convergence of this algorithm is spectacular: each iteration triples the number of digits of accuracy.

Theorem 27.3. *Rayleigh quotient iteration converges to an eigenvalue/eigenvector pair for all except a set of measure zero of starting vectors $v^{(0)}$. When it converges, the convergence is ultimately cubic in the sense that if λ_J is an eigenvalue of A and $v^{(0)}$ is sufficiently close to the eigenvector q_J, then*

$$\|v^{(k+1)} - (\pm q_J)\| = O(\|v^{(k)} - (\pm q_J)\|^3) \tag{27.6}$$

and

$$|\lambda^{(k+1)} - \lambda_J| = O(|\lambda^{(k)} - \lambda_J|^3) \tag{27.7}$$

as $k \to \infty$. The $\pm$ signs are not necessarily the same on the two sides of (27.6).

Proof. We shall not prove the assertion about convergence for almost all starting vectors. Here, however, is a proof that if convergence occurs, it is ultimately cubic. For simplicity, we assume that the eigenvalue λ_J is simple. By (27.3), if $\|v^{(k)} - q_J\| \le \epsilon$ for sufficiently small ϵ, then the Rayleigh quotient yields an eigenvalue estimate $\lambda^{(k)}$ with $|\lambda^{(k)} - \lambda_J| = O(\epsilon^2)$. By the argument used to prove Theorem 27.2, if we now take one step of inverse iteration to obtain a new $v^{(k+1)}$ from $v^{(k)}$ and $\lambda^{(k)}$, then

$$\|v^{(k+1)} - q_J\| = O(|\lambda^{(k)} - \lambda_J|\,\|v^{(k)} - q_J\|) = O(\epsilon^3).$$

Moreover, the constants implicit in the O symbols are uniform throughout sufficiently small neighborhoods of λ_J and q_J. Thus we have convergence in the following pattern:

$$\begin{array}{rcl} \|v^{(k)} - (\pm q_J)\| & & |\lambda^{(k)} - \lambda_J| \\ \epsilon & \to & O(\epsilon^2) \\ \downarrow & \swarrow & \\ O(\epsilon^3) & \to & O(\epsilon^6) \\ \downarrow & \swarrow & \\ O(\epsilon^9) & \to & O(\epsilon^{18}) \\ \vdots & & \vdots \end{array}$$

The estimates (27.6)–(27.7) follow from the uniformity just mentioned. □

Example 27.1. Cubic convergence is so fast that we must give a numerical example. Consider the symmetric matrix

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 4 \end{bmatrix},$$

and let $v^{(0)} = (1,1,1)^T/\sqrt{3}$ be the initial eigenvector estimate. When Rayleigh quotient iteration is applied to A, the following values $\lambda^{(k)}$ are computed by the first three iterations:

$$\lambda^{(0)} = 5, \quad \lambda^{(1)} = 5.2131\ldots, \quad \lambda^{(2)} = 5.214319743184\ldots.$$

The actual value of the eigenvalue corresponding to the eigenvector closest to $v^{(0)}$ is $\lambda = 5.214319743377$. After only three iterations, Rayleigh quotient iteration has produced a result accurate to ten digits. Three more iterations would increase this figure to about 270 digits, if our machine precision were high enough. □

Operation Counts

We close this lecture with a note on the amount of work required to execute each step of the three iterations we have described.

First, suppose $A \in \mathbb{R}^{m\times m}$ is a full matrix. Then each step of power iteration involves a matrix-vector multiplication, requiring $O(m^2)$ flops. Each step of inverse iteration involves the solution of a linear system, which might seem to require $O(m^3)$ flops, but this figure reduces to $O(m^2)$ if the matrix is processed in advance by LU or QR factorization or another method. In the case of Rayleigh quotient iteration, the matrix to be inverted changes at each step, and beating $O(m^3)$ flops per step is not so straightforward.

These figures improve greatly if A is tridiagonal. Now, all three iterations require just $O(m)$ flops per step. For the analogous iterations involving non-symmetric matrices, incidentally, we must deal with Hessenberg instead of tridiagonal structure, and this figure increases to $O(m^2)$.

Exercises

27.1. Let $A \in \mathbb{C}^{m\times m}$ be given, not necessarily hermitian. Show that a number $z \in \mathbb{C}$ is a Rayleigh quotient of A if and only if it is a diagonal entry of Q^*AQ for some unitary matrix Q. Thus Rayleigh quotients are just diagonal entries of matrices, once you transform orthogonally to the right coordinate system.

27.2. Again let $A \in \mathbb{C}^{m\times m}$ be arbitrary. The set of all Rayleigh quotients of A, corresponding to all nonzero vectors $x \in \mathbb{C}^m$, is known as the *field of values* or *numerical range* of A, a subset of the complex plane denoted by $W(A)$.

(a) Show that $W(A)$ contains the convex hull of the eigenvalues of A.

(b) Show that if A is normal, then $W(A)$ is equal to the convex hull of the eigenvalues of A.

27.3. Show that for a nonhermitian matrix $A \in \mathbb{C}^{m\times m}$, the Rayleigh quotient $r(x)$ gives an eigenvalue estimate whose accuracy is generally linear,

not quadratic. Explain what convergence rate this suggests for the Rayleigh quotient iteration applied to nonhermitian matrices.

27.4. Every real symmetric square matrix can be orthogonally diagonalized, and the developments of this lecture are invariant under orthogonal changes of coordinates. Thus it would have been sufficient to carry out each derivation of this lecture under the assumption that A is a diagonal matrix with entries ordered by decreasing absolute value. Making this assumption, describe the form taken by (27.4), (27.5), and Algorithm 27.3.

27.5. As mentioned in the text, inverse iteration depends on the solution of a system of equations that may be exceedingly ill-conditioned, with condition number on the order of $\epsilon_{\text{machine}}^{-1}$. We know that it is impossible in general to solve ill-conditioned systems accurately. Is this not a fatal flaw in the algorithm?

Show as follows that the answer is no—that ill-conditioning is not a problem in inverse iteration. Suppose A is a real symmetric matrix with one eigenvalue much smaller than the others in absolute value (without loss of generality, we are taking $\mu = 0$). Suppose v is a vector with components in the directions of all the eigenvectors $q_1, \ldots, q_m$ of A, and suppose $Aw = v$ is solved backward stably, yielding a computed vector $\tilde{w}$. Making use of the calculation on p. 95, show that although $\tilde{w}$ may be far from w, $\tilde{w}/\|\tilde{w}\|$ will not be far from $w/\|w\|$.

27.6. What happens to Figure 27.1 if two of the eigenvalues of A are equal?

Lecture 28. QR Algorithm without Shifts

The QR algorithm, dating to the early 1960s, is one of the jewels of numerical analysis. Here we show that in its simplest form, this algorithm can be viewed as a stable procedure for computing QR factorizations of the matrix powers A, A^2, $A^3, \ldots$.

The QR Algorithm

The most basic version of the QR algorithm seems impossibly simple.

Algorithm 28.1. "Pure" QR Algorithm

$A^{(0)} = A$
for $k = 1, 2, \ldots$
    $Q^{(k)} R^{(k)} = A^{(k-1)}$ QR factorization of $A^{(k-1)}$
    $A^{(k)} = R^{(k)} Q^{(k)}$ Recombine factors in reverse order

All we do is take a QR factorization, multiply the computed factors Q and R together in the reverse order RQ, and repeat. Yet under suitable assumptions, this simple algorithm converges to a Schur form for the matrix A—upper-triangular if A is arbitrary, diagonal if A is hermitian. Here, to keep the discussion simple, we shall continue to assume as in the last lecture that A is real and symmetric, with real eigenvalues λ_j and orthonormal eigenvectors q_j. Thus our interest is in the convergence of the matrices $A^{(k)}$ to diagonal form.

For convergence to diagonal form to be useful for finding eigenvalues, of course, the operations involved must be similarity transformations. This is easily verified: the QR algorithm first triangularizes $A^{(k)}$ by forming $R^{(k)} = (Q^{(k)})^T A^{(k-1)}$, and the multiplication on the right by $Q^{(k)}$ then gives $A^{(k)} = (Q^{(k)})^T A^{(k-1)} Q^{(k)}$. In fact, we have seen this similarity transformation before: it is the "bad idea" mentioned in Lecture 26. Although this transformation is a bad idea when trying to reduce A to triangular form in a single step, it turns out to be quite powerful as the basis of an iteration.

Like the Rayleigh quotient iteration, the QR algorithm for real symmetric matrices converges cubically. To achieve this performance, however, the algorithm as presented above must be modified by the introduction of shifts at each step. The use of shifts is one of three modifications of Algorithm 28.1 that are required to bring it closer to a practical algorithm:

1. Before starting the iteration, A is reduced to tridiagonal form, as discussed in Lecture 26.
2. Instead of $A^{(k)}$, a shifted matrix $A^{(k)} - \mu^{(k)} I$ is factored at each step, where $\mu^{(k)}$ is some eigenvalue estimate.
3. Whenever possible, and in particular whenever an eigenvalue is found, the problem is "deflated" by breaking $A^{(k)}$ into submatrices.

A QR algorithm incorporating these modifications has the following outline.

Algorithm 28.2. "Practical" QR Algorithm

$(Q^{(0)})^T A^{(0)} Q^{(0)} = A$ — $A^{(0)}$ is a tridiagonalization of A

for $k = 1, 2, \ldots$

- Pick a shift $\mu^{(k)}$ — e.g., choose $\mu^{(k)} = A^{(k-1)}_{mm}$
- $Q^{(k)} R^{(k)} = A^{(k-1)} - \mu^{(k)} I$ — QR factorization of $A^{(k-1)} - \mu^{(k)} I$
- $A^{(k)} = R^{(k)} Q^{(k)} + \mu^{(k)} I$ — Recombine factors in reverse order
- If any off-diagonal element $A^{(k)}_{j,j+1}$ is sufficiently close to zero,
  - set $A_{j,j+1} = A_{j+1,j} = 0$ to obtain
  $$\begin{bmatrix} A_1 & 0 \\ 0 & A_2 \end{bmatrix} = A^{(k)}$$
  - and now apply the QR algorithm to A_1 and A_2.

This algorithm, the QR algorithm with well-chosen shifts, has been the standard method for computing all the eigenvalues of a matrix since the early 1960s. Only in the 1990s has a competitor emerged, the divide-and-conquer algorithm described in Lecture 30.

Tridiagonalization was discussed in Lecture 26, shifts are discussed in the next lecture, and deflation is not discussed further in this book. For now, let

us confine our attention to the "pure" QR algorithm and explain how it finds eigenvalues.

Unnormalized Simultaneous Iteration

Our approach will be to relate the QR algorithm to another method called *simultaneous iteration*, whose behavior is more obvious.

The idea of simultaneous iteration is to apply the power iteration to several vectors at once. (An equivalent term is *block power iteration*.) Suppose we start with a set of n linearly independent vectors $v_1^{(0)}, \ldots, v_n^{(0)}$. It seems plausible that just as $A^k v_1^{(0)}$ converges as $k \to \infty$ (under suitable assumptions) to the eigenvector corresponding to the largest eigenvalue of A in absolute value, the space $\langle A^k v_1^{(0)}, \ldots, A^k v_n^{(0)} \rangle$ should converge (again under suitable assumptions) to the space $\langle q_1, \ldots, q_n \rangle$ spanned by the eigenvectors $q_1, \ldots, q_n$ of A corresponding to the n largest eigenvalues in absolute value.

In matrix notation, we might proceed like this. Define $V^{(0)}$ to be the $m \times n$ initial matrix

$$V^{(0)} = \left[\begin{array}{c|c|c} & & \\ v_1^{(0)} & \cdots & v_n^{(0)} \\ & & \end{array} \right], \tag{28.1}$$

and define $V^{(k)}$ to be the result after k applications of A:

$$V^{(k)} = A^k V^{(0)} = \left[\begin{array}{c|c|c} & & \\ v_1^{(k)} & \cdots & v_n^{(k)} \\ & & \end{array} \right]. \tag{28.2}$$

Since our interest is in the column space of $V^{(k)}$, let us extract a well-behaved basis for this space by computing a reduced QR factorization of $V^{(k)}$:

$$\hat{Q}^{(k)} \hat{R}^{(k)} = V^{(k)}. \tag{28.3}$$

Here $\hat{Q}^{(k)}$ and $\hat{R}^{(k)}$ have dimensions $m \times n$ and $n \times n$, respectively. It seems plausible that as $k \to \infty$, under suitable assumptions, the successive columns of $\hat{Q}^{(k)}$ should converge to the eigenvectors $\pm q_1, \pm q_2, \ldots, \pm q_n$.

This expectation can be justified by an analysis analogous to that of the last lecture. If we expand $v_j^{(0)}$ and $v_j^{(k)}$ in the eigenvectors of A, we have

$$\begin{aligned} v_j^{(0)} &= a_{1j} q_1 + \cdots + a_{mj} q_m, \\ v_j^{(k)} &= \lambda_1^k a_{1j} q_1 + \cdots + \lambda_m^k a_{mj} q_m. \end{aligned}$$

As in the last section, simple convergence results will hold provided that two conditions are satisfied. The first assumption we make is that the leading $n+1$ eigenvalues are distinct in absolute value:

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > |\lambda_{n+1}| \geq |\lambda_{n+2}| \geq \cdots \geq |\lambda_m|. \tag{28.4}$$

Our second assumption is that the collection of expansion coefficients a_{ij} is in an appropriate sense nonsingular. Define $\hat{Q}$ to be the $m \times n$ matrix whose columns are the eigenvectors $q_1, q_2, \ldots, q_n$. (Thus $\hat{Q}$, a matrix of eigenvectors, is entirely different from $\hat{Q}^{(k)}$, a factor in a reduced QR factorization.) We assume the following:

All the leading principal submatrices of $\hat{Q}^T V^{(0)}$ are nonsingular. (28.5)

By the leading principal submatrices of $\hat{Q}^T V^{(0)}$, we mean its upper-left square submatrices of dimensions 1×1, 2×2, $\ldots$, $n\times n$. (The condition (28.5) happens to be equivalent to the condition that $\hat{Q}^T V^{(0)}$ has an LU factorization; see Exercise 20.1.)

Theorem 28.1. *Suppose that the iteration* (28.1)–(28.3) *is carried out and that assumptions* (28.4) *and* (28.5) *are satisfied. Then as $k \to \infty$, the columns of the matrices $\hat{Q}^{(k)}$ converge linearly to the eigenvectors of A:*

$$\|q_j^{(k)} - \pm q_j\| = O(C^k) \tag{28.6}$$

for each j with $1 \le j \le n$, where $C < 1$ is the constant $\max_{1\le k\le n} |\lambda_{k+1}|/|\lambda_k|$. As in the theorems of the last lecture, the $\pm$ sign means that at each step k, one or the other choice of sign is to be taken, and then the indicated bound holds.

Proof. Extend $\hat{Q}$ to a full $m\times m$ orthogonal matrix Q of eigenvectors of A, and let Λ be the corresponding diagonal matrix of eigenvalues; thus $A = Q\Lambda Q^T$. Just as $\hat{Q}$ is the leading $m \times n$ section of Q, define $\hat{\Lambda}$ (still diagonal) to be the leading $n \times n$ section of Λ. Then we have

$$V^{(k)} = A^k V^{(0)} = Q\Lambda^k Q^T V^{(0)} = \hat{Q}\hat{\Lambda}^k \hat{Q}^T V^{(0)} + O(|\lambda_{n+1}|^k)$$

as $k \to \infty$. If (28.5) holds, then in particular, $\hat{Q}^T V^{(0)}$ is nonsingular, so we can multiply the term $O(|\lambda_{n+1}|^k)$ on the right by $(\hat{Q}^T V^{(0)})^{-1}\hat{Q}^T V^{(0)}$ to transform this equation to

$$V^{(k)} = \left(\hat{Q}\hat{\Lambda}^k + O(|\lambda_{n+1}|^k)\right)\hat{Q}^T V^{(0)}.$$

Since $\hat{Q}^T V^{(0)}$ is nonsingular, the column space of this matrix is the same as the column space of

$$\hat{Q}\hat{\Lambda}^k + O(|\lambda_{n+1}|^k).$$

From the form of $\hat{Q}\hat{\Lambda}^k$ and the assumption (28.4), it is clear that this column space converges linearly to that of $\hat{Q}$. This convergence can be quantified, for example, by defining angles between subspaces; we omit the details.

Now in fact, we have assumed that not only is $\hat{Q}^T V^{(0)}$ nonsingular, but so are all of its leading principal submatrices. It follows that the argument above also applies to leading subsets of the columns of $V^{(k)}$ and $\hat{Q}$: the first columns,

the first and second columns, the first and second and third columns, and so on. In each case we conclude that the space spanned by the indicated columns of $V^{(k)}$ converges linearly to the space spanned by the corresponding columns of $\hat{Q}$. From this convergence of all the successive column spaces, together with the definition of the QR factorization (28.3), (28.6) follows. □

Simultaneous Iteration

As $k \to \infty$, the vectors $v_1^{(k)}, \ldots, v_n^{(k)}$ in the algorithm (28.1)–(28.3) all converge to multiples of the same dominant eigenvector q_1 of A. Thus, although the space they span, $\langle v_1^{(k)}, \ldots, v_j^{(k)} \rangle$, converges to something useful, these vectors constitute a highly ill-conditioned basis of that space. If we actually carried out simultaneous iteration in floating point arithmetic as just described, the desired information would quickly be lost to rounding errors.

The remedy is simple: one must orthonormalize at each step rather than once and for all. Thus we shall not construct $V^{(k)}$ as defined above, but a different sequence of matrices $Z^{(k)}$ with the same column spaces.

Algorithm 28.3. Simultaneous Iteration

Pick $\hat{Q}^{(0)} \in \mathbb{R}^{m \times n}$ with orthonormal columns.
for $k = 1, 2, \ldots$
$\quad Z = A\hat{Q}^{(k-1)}$
$\quad \hat{Q}^{(k)}\hat{R}^{(k)} = Z$ reduced QR factorization of Z

From the form of this algorithm, it is clear that the column spaces of $\hat{Q}^{(k)}$ and $Z^{(k)}$ are the same, both being equal to the column space of $A^k\hat{Q}^{(0)}$. Thus, mathematically speaking, this new formulation of simultaneous iteration converges under the same conditions as the old one.

Theorem 28.2. *Algorithm* 28.3 *generates the same matrices* $\hat{Q}^{(k)}$ *as the iteration* (28.1)–(28.3) *considered in Theorem* 28.1 *(assuming that the initial matrices* $\hat{Q}^{(0)}$ *are the same), and under the same assumptions* (28.4) *and* (28.5), *it converges as described in that theorem.*

Simultaneous Iteration $\Longleftrightarrow$ QR Algorithm

Now we can explain the QR algorithm. It is equivalent to simultaneous iteration applied to a full set of $n = m$ initial vectors, namely, the identity, $\hat{Q}^{(0)} = I$. Since the matrices $\hat{Q}^{(k)}$ are now square, we are dealing with full QR factorizations and can drop the hats on $\hat{Q}^{(k)}$ and $\hat{R}^{(k)}$. In fact, we shall

replace $\hat{R}^{(k)}$ by $R^{(k)}$ but $\hat{Q}^{(k)}$ by $\underline{Q}^{(k)}$ in order to distinguish the Q matrices of simultaneous iteration from those of the QR algorithm.

Here are the three formulas that define simultaneous iteration with $\underline{Q}^{(0)} = I$, followed by a fourth formula that we shall take as a definition of an $m \times m$ matrix $A^{(k)}$:

Simultaneous Iteration:

$$\underline{Q}^{(0)} = I, \tag{28.7}$$

$$Z = A\underline{Q}^{(k-1)}, \tag{28.8}$$

$$Z = \underline{Q}^{(k)} R^{(k)}, \tag{28.9}$$

$$A^{(k)} = (\underline{Q}^{(k)})^T A \underline{Q}^{(k)}. \tag{28.10}$$

And here are the three formulas that define the pure QR algorithm, followed by a fourth formula that we shall take as a definition of an $m \times m$ matrix $\underline{Q}^{(k)}$:

Unshifted QR Algorithm:

$$A^{(0)} = A, \tag{28.11}$$

$$A^{(k-1)} = Q^{(k)} R^{(k)}, \tag{28.12}$$

$$A^{(k)} = R^{(k)} Q^{(k)}, \tag{28.13}$$

$$\underline{Q}^{(k)} = Q^{(1)} Q^{(2)} \cdots Q^{(k)}. \tag{28.14}$$

Additionally, for both algorithms, let us define one further $m \times m$ matrix $\underline{R}^{(k)}$,

$$\underline{R}^{(k)} = R^{(k)} R^{(k-1)} \cdots R^{(1)}. \tag{28.15}$$

We can now exhibit the equivalence of these two algorithms.

Theorem 28.3. *The processes* (28.7)–(28.10) *and* (28.11)–(28.14) *generate identical sequences of matrices* $\underline{R}^{(k)}$, $\underline{Q}^{(k)}$, *and* $A^{(k)}$, *namely, those defined by the QR factorization of the kth power of* A,

$$A^k = \underline{Q}^{(k)} \underline{R}^{(k)}, \tag{28.16}$$

together with the projection

$$A^{(k)} = (\underline{Q}^{(k)})^T A \underline{Q}^{(k)}. \tag{28.17}$$

Proof. We proceed by induction in k. The base case $k = 0$ is trivial. For both simultaneous iteration and the QR algorithm, equations (28.7)–(28.15) imply $A^0 = \underline{Q}^{(0)} = \underline{R}^{(0)} = I$ and $A^{(0)} = A$, from which (28.16) and (28.17) are immediate.

Consider now the case $k \geq 1$ for simultaneous iteration. Formula (28.17) is valid by virtue of the definition (28.10) (they are identical), so we need only verify (28.16), which can be done as follows:

$$A^k = A\underline{Q}^{(k-1)}\underline{R}^{(k-1)} = \underline{Q}^{(k)}R^{(k)}\underline{R}^{(k-1)} = \underline{Q}^{(k)}\underline{R}^{(k)}.$$

The first equality follows from the inductive hypothesis on (28.16), the second from (28.8) and (28.9), and the third from (28.15).

On the other hand, consider the case $k \geq 1$ for the QR algorithm. We can verify (28.16) by the sequence

$$A^k = A\underline{Q}^{(k-1)}\underline{R}^{(k-1)} = \underline{Q}^{(k-1)}A^{(k-1)}\underline{R}^{(k-1)} = \underline{Q}^{(k)}\underline{R}^{(k)}.$$

The first equality follows from the inductive hypothesis on (28.16), the second from the inductive hypothesis on (28.17), and the third from (28.12), together with (28.14) and (28.15). Finally, we can verify (28.17) by the sequence

$$A^{(k)} = (Q^{(k)})^T A^{(k-1)} Q^{(k)} = (\underline{Q}^{(k)})^T A \underline{Q}^{(k)}.$$

The first equality follows from (28.12) and (28.13), and the second from the inductive hypothesis on (28.17). □

Convergence of the QR Algorithm

All the pieces are in place. We can now say a great deal about the convergence of the unshifted QR algorithm.

First, at the level of qualitative understanding: (28.16) and (28.17) are the key. The first of these explains why the QR algorithm can be expected to find eigenvectors: it constructs orthonormal bases for successive powers A^k. The second explains why the algorithm finds eigenvalues. From (28.17) it follows that the diagonal elements of $A^{(k)}$ are Rayleigh quotients of A corresponding to the columns of $\underline{Q}^{(k)}$ (see Exercise 27.1). As those columns converge to eigenvectors, the Rayleigh quotients converge (twice as fast, by (27.3)) to the corresponding eigenvalues. Meanwhile, (28.17) implies that the off-diagonal elements of $A^{(k)}$ correspond to generalized Rayleigh quotients involving approximations of distinct eigenvectors of A on the left and the right. Since these approximations must become orthogonal as they converge to distinct eigenvectors, the off-diagonal elements of $A^{(k)}$ must converge to zero.

We cannot emphasize too strongly how fundamental equations (28.16) and (28.17) are to an understanding of the unshifted QR algorithm. They are memorable; and from them, everything of importance follows.

As for a more quantitative understanding, we have the following consequence of Theorem 28.2.

Theorem 28.4. *Let the pure QR algorithm (Algorithm* 28.1*) be applied to a real symmetric matrix A whose eigenvalues satisfy $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_m|$ and whose corresponding eigenvector matrix Q has all nonsingular leading principal submatrices. Then as $k \to \infty$, $A^{(k)}$ converges linearly with constant $\max_j |\lambda_{j+1}|/|\lambda_j|$ to $\mathrm{diag}(\lambda_1, \ldots, \lambda_m)$, and $\underline{Q}^{(k)}$ (with the signs of its columns adjusted as necessary) converges at the same rate to Q.*

Exercises

28.1. What happens if you apply the unshifted QR algorithm to an orthogonal matrix? Figure out the answer, and then explain how it relates to Theorem 28.4.

28.2. The preliminary reduction to tridiagonal form would be of little use if the steps of the QR algorithm did not preserve this structure. Fortunately, they do.

(a) In the QR factorization $A = QR$ of a symmetric tridiagonal matrix A, which entries of R are in general nonzero? Which entries of Q? (In practice we do not form Q explicitly.)

(b) Show that the tridiagonal structure is recovered when the product RQ is formed.

(c) Explain how Givens rotations or 2×2 Householder reflections can be used in the computation of the QR factorization of a tridiagonal matrix, reducing the operation count far below what would be required for a full matrix.

28.3. A real symmetric matrix A has an eigenvalue 1 of multiplicity 8, while all the rest of the eigenvalues are ≤ 0.1 in absolute value. Describe an algorithm for finding an orthonormal basis of the 8-dimensional eigenspace corresponding to the dominant eigenvalue.

28.4. Consider one step of Algorithm 28.1 applied to a tridiagonal symmetric matrix $A \in \mathbb{R}^{m \times m}$.

(a) If only eigenvalues are desired, then only $A^{(k)}$ is needed at step k, not $Q^{(k)}$. Determine how many flops are required to get from $A^{(k-1)}$ to $A^{(k)}$ using standard methods described in this book.

(b) If all the eigenvectors are desired, then the matrix $\underline{Q}^{(k)} = Q^{(1)}Q^{(2)} \cdots Q^{(k)}$ will need to be accumulated too. Determine how many flops are now required to get from step $k-1$ to step k.

Lecture 29. QR Algorithm with Shifts

What makes the QR iteration fly is the introduction of shifts $A \to A - \mu I$ at each step. Here we explain how this idea leads to cubic convergence, thanks to an implicit connection with the Rayleigh quotient iteration.

Connection with Inverse Iteration

We continue to assume that $A \in \mathbb{R}^{m \times m}$ is real and symmetric, with real eigenvalues $\{\lambda_j\}$ and orthonormal eigenvectors $\{q_j\}$.

As we have seen, the "pure" QR algorithm (Algorithm 28.1) is equivalent to simultaneous iteration applied to the identity matrix, and in particular, the first column of the result evolves according to the power iteration applied to e_1. There is a dual to this observation. Algorithm 28.1 is also equivalent to *simultaneous inverse iteration* applied to a "flipped" identity matrix P, and in particular, the mth column of the result evolves according to inverse iteration applied to e_m.

We can establish this claim as follows. Let $Q^{(k)}$, as in the last lecture, be the orthogonal factor at the kth step of the QR algorithm. In the last lecture, we showed that the accumulated product (28.14) of these matrices,

$$\underline{Q}^{(k)} = \prod_{j=1}^{k} Q^{(j)} = \left[\begin{array}{c|c|c|c} & & & \\ q_1^{(k)} & q_2^{(k)} & \cdots & q_m^{(k)} \\ & & & \end{array} \right],$$

is the same orthogonal matrix that appears at step k (28.9) of simultaneous iteration. Another way to put this was to say that $\underline{Q}^{(k)}$ is the orthogonal factor in a QR factorization (28.16),

$$A^k = \underline{Q}^{(k)} \underline{R}^{(k)}.$$

Now consider what happens if we invert this formula. We calculate

$$A^{-k} \;=\; (\underline{R}^{(k)})^{-1} \underline{Q}^{(k)T} \;=\; \underline{Q}^{(k)} (\underline{R}^{(k)})^{-T}; \tag{29.1}$$

for the second equality we have used the fact that A^{-1} is symmetric. Let P denote the $m \times m$ permutation matrix that reverses row or column order:

$$P = \begin{bmatrix} & & & 1 \\ & & 1 & \\ & \cdots & & \\ 1 & & & \end{bmatrix}.$$

Since $P^2 = I$, (29.1) can be rewritten as

$$A^{-k}P \;=\; [\underline{Q}^{(k)}P][P(\underline{R}^{(k)})^{-T}P]. \tag{29.2}$$

The first factor in this product, $\underline{Q}^{(k)}P$, is orthogonal. The second, $P(\underline{R}^{(k)})^{-T}P$, is upper-triangular (start with the lower-triangular matrix $(\underline{R}^{(k)})^{-T}$, flip it top-to-bottom, then flip it again left-to-right). Thus (29.2) can be interpreted as a QR factorization of $A^{-k}P$. In other words, we are effectively carrying out simultaneous iteration on A^{-1} applied to the initial matrix P, which is to say, simultaneous inverse iteration on A. In particular, the first column of $\underline{Q}^{(k)}P$—the last column of $\underline{Q}^{(k)}$—is the result of applying k steps of inverse iteration to the vector e_m.

Connection with Shifted Inverse Iteration

Thus the QR algorithm is both simultaneous iteration and simultaneous inverse iteration: the symmetry is perfect. But, as we saw in Lecture 27, there is a huge difference between power iteration and inverse iteration: the latter can be accelerated arbitrarily through the use of shifts. The better we can estimate an eigenvalue $\mu \approx \lambda_J$, the more we shall accomplish by a step of inverse iteration with the shifted matrix $A - \mu I$. Algorithm 28.2 showed how shifts are introduced into a step of the QR algorithm. Doing this corresponds exactly to shifts in the corresponding simultaneous iteration and inverse iteration processes, and their beneficial effect is therefore exactly the same.

Let $\mu^{(k)}$ denote the eigenvalue estimate chosen at the kth step of the QR algorithm. From Algorithm 28.2, the relationship between steps $k-1$ and k of the shifted QR algorithm is

$$\begin{aligned} A^{(k-1)} - \mu^{(k)} I &= Q^{(k)} R^{(k)}, \\ A^{(k)} &= R^{(k)} Q^{(k)} + \mu^{(k)} I. \end{aligned}$$

This implies

$$A^{(k)} = (Q^{(k)})^T A^{(k-1)} Q^{(k)}, \tag{29.3}$$

and by induction,

$$A^{(k)} = (\underline{Q}^{(k)})^T A \underline{Q}^{(k)},$$

which is unchanged from (28.17). However, (28.16) no longer holds. Instead, we have the factorization

$$(A - \mu^{(k)} I)(A - \mu^{(k-1)} I) \cdots (A - \mu^{(1)} I) \;=\; \underline{Q}^{(k)} \underline{R}^{(k)}, \tag{29.4}$$

a shifted variation on simultaneous iteration (we omit the proof). In words, $\underline{Q}^{(k)} = \prod_{j=1}^{k} Q^{(j)}$ is an orthogonalization of $\prod_{j=k}^{1}(A - \mu^{(j)} I)$. The first column of $\underline{Q}^{(k)}$ is the result of applying shifted power iteration to e_1 using the shifts $\mu^{(j)}$, and the last column is the result of applying k steps of shifted inverse iteration to e_m with the same shifts. If the shifts are good eigenvalue estimates, this last column of $\underline{Q}^{(k)}$ converges quickly to an eigenvector.

Connection with Rayleigh Quotient Iteration

We have discovered a powerful tool hidden in the shifted QR algorithm: shifted inverse iteration. To complete the idea, we now need a way of choosing shifts to achieve fast convergence in the last column of $\underline{Q}^{(k)}$.

The Rayleigh quotient is a good place to start. To estimate the eigenvalue corresponding to the eigenvector approximated by the last column of $\underline{Q}^{k}$, it is natural to apply the Rayleigh quotient to this last column. This gives us

$$\mu^{(k)} \;=\; \frac{(q_m^{(k)})^T A\, q_m^{(k)}}{(q_m^{(k)})^T q_m^{(k)}} \;=\; (q_m^{(k)})^T A\, q_m^{(k)}. \tag{29.5}$$

If this number is chosen as the shift at every step, the eigenvalue and eigenvector estimates $\mu^{(k)}$ and $q_m^{(k)}$ are identical to those that are computed by the Rayleigh quotient iteration starting with e_m. Therefore, the QR algorithm has cubic convergence in the sense that $q_m^{(k)}$ converges cubically to an eigenvector.

Notice that, in the QR algorithm, the Rayleigh quotient $r(q_m^{(k)})$ appears as the m, m entry of $A^{(k)}$—so it comes for free! We mentioned this at the end of the last lecture, but here is an explicit derivation for emphasis. Starting with (29.3), we have

$$A_{mm}^{(k)} = e_m^T A^{(k)} e_m = e_m^T \underline{Q}^{(k)T} A \underline{Q}^{(k)} e_m = q_m^{(k)T} A q_m^{(k)}. \tag{29.6}$$

Therefore, (29.5) is the same as simply setting $\mu^{(k)} = A_{mm}^{(k)}$. This is known as the *Rayleigh quotient shift*.

Wilkinson Shift

Although the Rayleigh quotient shift gives cubic convergence in the generic case, convergence is not guaranteed for all initial conditions. We can see this with a simple example. Consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{29.7}$$

The unshifted QR algorithm does not converge at all:

$$A = Q^{(1)}R^{(1)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

$$A^{(1)} = R^{(1)}Q^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = A.$$

The Rayleigh quotient shift $\mu = A_{mm}$, however, has no effect either, since $A_{mm} = 0$. Thus it is clear that in the worst case, the QR algorithm with the Rayleigh quotient shift may fail.

The problem arises because of the symmetry of the eigenvalues. One eigenvalue is $+1$, and the other is -1, so when we attempt to improve the eigenvalue estimate 0, the tendency to favor each eigenvalue is equal, and the estimate is not improved. What is needed is an eigenvalue estimate that can break the symmetry. One such choice is defined as follows. Let B denote the lower-rightmost 2×2 submatrix of $A^{(k)}$:

$$B = \begin{bmatrix} a_{m-1} & b_{m-1} \\ b_{m-1} & a_m \end{bmatrix}.$$

The *Wilkinson shift* is defined as that eigenvalue of B that is closer to a_m, where in the case of a tie, one of the two eigenvalues of B is chosen arbitrarily. A numerically stable formula for the Wilkinson shift is

$$\mu = a_m - \operatorname{sign}(\delta)\, b_{m-1}^2 \Big/ \left(|\delta| + \sqrt{\delta^2 + b_{m-1}^2} \right), \tag{29.8}$$

where $\delta = (a_{m-1} - a_m)/2$. If $\delta = 0$, $\operatorname{sign}(\delta)$ can be arbitrarily set equal to 1 or -1.

Like the Rayleigh quotient shift, the Wilkinson shift achieves cubic convergence in the generic case. Moreover, it can be shown that it achieves at least quadratic convergence in the worst case. In particular, the QR algorithm with the Wilkinson shift always converges (in exact arithmetic).

In the example (29.7), the Wilkinson shift is either $+1$ or -1. Thus the symmetry is broken, and convergence takes place in one step.

Stability and Accuracy

This completes our discussion of the mechanics of the QR algorithm, though many practical details have been omitted, such as conditions for deflation and "implicit" strategies for shifting. It remains to say a word about stability and accuracy.

As one might expect from its use of orthogonal matrices, the QR algorithm is backward stable. As in previous lectures, the simplest way to formulate this result is to let $\tilde{\Lambda}$ denote the diagonalization of A as computed in floating point arithmetic, and $\tilde{Q}$ the exactly orthogonal matrix associated with the product of all the numerically computed Householder reflections (or Givens rotations) utilized along the way. Here is what can be proved.

Theorem 29.1. *Let a real, symmetric, tridiagonal matrix $A \in \mathbb{R}^{m\times m}$ be diagonalized by the QR algorithm (Algorithm 28.2) on a computer satisfying* (13.5) *and* (13.7)*, and let $\tilde{\Lambda}$ and $\tilde{Q}$ be defined as indicated above. Then we have*

$$\tilde{Q}\tilde{\Lambda}\tilde{Q}^* = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}) \tag{29.9}$$

for some $\delta A \in \mathbb{C}^{m\times m}$.

Like most of the algorithms in this book, then, the QR algorithm produces an exact solution of a slightly perturbed problem. Combining Theorems 26.1 and 29.1, we see that tridiagonal reduction followed by the QR algorithm is a backward stable algorithm for computing eigenvalues of matrices. To see what this implies about accuracy of the computed eigenvalues, we may combine this conclusion with the result (26.4) concerning perturbation of eigenvalues of real symmetric matrices (a special case of normal matrices). The conclusion is that the computed eigenvalues $\tilde{\lambda}_j$ satisfy

$$\frac{|\tilde{\lambda}_j - \lambda_j|}{\|A\|} \;=\; O(\epsilon_{\text{machine}}). \tag{29.10}$$

This is not a bad result at all for an algorithm that requires just $\sim \frac{4}{3}m^3$ flops, two-thirds the cost of computing the product of a pair of $m \times m$ matrices!

Exercise

29.1. This five-part problem asks you to put together a MATLAB program that finds all the eigenvalues of a real symmetric matrix, using only elementary building blocks. It is not necessary to achieve optimal constant factors by exploiting symmetry or zero structure optimally. It is possible to solve the whole problem by a program about fifty lines long.

(a) Write a function `T = tridiag(A)` that reduces a real symmetric $m \times m$ matrix to tridiagonal form by orthogonal similarity transformations. Your program should use only elementary MATLAB operations—not the built-in function `hess`, for example. Your output matrix `T` should be symmetric and tridiagonal up to rounding errors. If you like, add a line that forces `T` at the end to be exactly symmetric and tridiagonal. For an example, apply your program to `A = hilb(4)`.

(b) Write a function `Tnew = qralg(T)` that runs the unshifted QR algorithm on a real tridiagonal matrix `T`. For the QR factorization at each step, use programs `[W,R] = house(A)` and `Q = formQ(W)` of Exercise 10.2 if available, or MATLAB's command `qr`, or, for greater efficiency, a new code based on Givens rotations or 2×2 Householder reflections rather than $m \times m$ operations. Again, you may wish to enforce symmetry and tridiagonality at each step. Your program should stop and return the current tridiagonal matrix `T` as `Tnew` when the $m, m-1$ element satisfies $|t_{m,m-1}| < 10^{-12}$ (hardly an industrial strength convergence criterion!). Again, apply your program to `A = hilb(4)`.

(c) Write a driver program which (i) calls `tridiag`, (ii) calls `qralg` to get one eigenvalue, (iii) calls `qralg` with a smaller matrix to get another eigenvalue, and so on until all of the eigenvalues of A are determined. Set things up so that the values of $|t_{m,m-1}|$ at every QR iteration are stored in a vector and so that at the end, your program generates a semilogy plot of these values as a function of the number of QR factorizations. (Here m will step from `length(A)` to `length(A)`-1 and so on down to 3 and finally 2 as the deflation proceeds, and the plot will be correspondingly sawtoothed.) Run your program for `A = hilb(4)`. The output should be a set of eigenvalues and a "sawtooth plot."

(d) Modify `qralg` so that it uses the Wilkinson shift at each step. Turn in the new sawtooth plot for the same example.

(e) Rerun your program for the matrix `A = diag(15:-1:1) + ones(15,15)` and generate two sawtooth plots corresponding to shift and no shift. Discuss the rates of convergence observed here and for the earlier matrix. Is the convergence linear, superlinear, quadratic, cubic . . . ? Is it meaningful to speak of a certain "number of QR iterations per eigenvalue?"

Lecture 30. Other Eigenvalue Algorithms

There is more to the computation of eigenvalues than the QR algorithm. In this lecture we briefly mention three famous alternatives for real symmetric eigenvalue problems: the Jacobi algorithm, for full matrices, and the bisection and divide-and-conquer algorithms, for tridiagonal matrices.

Jacobi

One of the oldest ideas for computing eigenvalues of matrices is the *Jacobi algorithm*, introduced by Jacobi in 1845. This method has attracted attention throughout the computer era, especially since the advent of parallel computing, though it has never quite managed to displace the competition.

The idea is as follows. For matrices of dimension 5 or larger, we know that eigenvalues can only be obtained by iteration (Lecture 25). However, smaller matrices than this can be handled in one step. Why not diagonalize a small submatrix of A, then another, and so on, hoping eventually to converge to a diagonalization of the full matrix?

The idea has been tried with 4×4 submatrices, but the standard approach is based on 2×2 submatrices. A 2×2 real symmetric matrix can be diagonalized in the form

$$J^T \begin{bmatrix} a & d \\ d & b \end{bmatrix} J = \begin{bmatrix} \neq 0 & 0 \\ 0 & \neq 0 \end{bmatrix}, \tag{30.1}$$

where J is orthogonal. Now there are several ways to choose J. One could take it to be a 2×2 Householder reflection of the form

$$F = \begin{bmatrix} -c & s \\ s & c \end{bmatrix}, \tag{30.2}$$

where $s = \sin\theta$ and $c = \cos\theta$ for some θ. Note that $\det F = -1$, the hallmark of a reflection. Alternatively, one can use not a reflection but a rotation,

$$J = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \tag{30.3}$$

with $\det J = 1$. This is the standard approach for the Jacobi algorithm. It can be shown that the diagonalization (30.1) is accomplished if θ satisfies

$$\tan(2\theta) = \frac{2d}{b-a}, \tag{30.4}$$

and the matrix J based on this choice is called a *Jacobi rotation*. (It has the same form as a Givens rotation (Exercise 10.4); the only difference is that θ is chosen to make J^TAJ diagonal rather than J^TA triangular.)

Now let $A \in \mathbb{R}^{m\times m}$ be symmetric. The Jacobi algorithm consists of the iterative application of transformations (30.1) based on matrices defined by (30.3) and (30.4). The matrix J is now enlarged to an $m \times m$ matrix that is the identity in all but four entries, where it has the form (30.3). Applying J^T on the left modifies two rows of A, and applying J on the right modifies two columns. At each step a symmetric pair of zeros is introduced into the matrix, but previous zeros are destroyed. Just as with the QR algorithm, however, the usual effect is that the magnitudes of these nonzeros shrink steadily.

Which off-diagonal entries a_{ij} should be zeroed at each step? The approach naturally fitted to hand computation is to pick the largest off-diagonal entry at each step. Analysis of convergence then becomes a triviality, for one can show that the sum of the squares of the off-diagonal entries decreases by at least the factor $1 - 2/(m^2 - m)$ at each step (Exercise 30.3). After $O(m^2)$ steps, each requiring $O(m)$ operations, the sum of squares must drop by a constant factor, and convergence to accuracy $\epsilon_{\text{machine}}$ is assured after $O(m^3 \log(\epsilon_{\text{machine}}))$ operations. In fact, it is known that the convergence is better than this, ultimately quadratic rather than linear, so the actual operation count is $O(m^3 \log(|\log(\epsilon_{\text{machine}})|))$ (Exercise 25.2).

On a computer, the off-diagonal entries are generally eliminated in a cyclic manner that avoids the $O(m^2)$ search for the largest. For example, if the $m(m-1)/2$ superdiagonal entries are eliminated in the simplest row-wise order, beginning with $a_{12}, a_{13}, \ldots$, then rapid asymptotic convergence is again guaranteed. After one *sweep* of 2×2 operations involving all of the $m(m-1)/2$ pairs of off-diagonal entries, the accuracy has generally improved by better than a constant factor, and again, the convergence is ultimately quadratic.

The Jacobi method is attractive because it deals only with pairs of rows and columns at a time, making it easily parallelizable (Exercise 30.4). The matrix is not tridiagonalized in advance; the Jacobi rotations would destroy that structure. Convergence for matrices of dimension $m \leq 1000$ is typically achieved in fewer than ten sweeps, and the final componentwise accuracy is generally even better than can be achieved by the QR algorithm. Unfortunately, even on parallel machines, the Jacobi algorithm is not usually as fast as tridiagonalization followed by the QR or divide-and-conquer algorithm (discussed below), though it usually comes within a factor of 10 (Exercise 30.2).

Bisection

Our next eigenvalue algorithm, the method of *bisection*, is of great practical importance. After a symmetric matrix has been tridiagonalized, this is the standard next step if one does not want all of the eigenvalues but just a subset of them. For example, bisection can find the largest 10% of the eigenvalues, or the smallest thirty eigenvalues, or all the eigenvalues in the interval $[1, 2]$. Once the desired eigenvalues are found, the corresponding eigenvectors can be obtained by one step of inverse iteration (Algorithm 27.2).

The starting point is elementary. Since the eigenvalues of a real symmetric matrix are real, we can find them by searching the real line for roots of the polynomial $p(x) = \det(A - xI)$. This sounds like a bad idea, for did we not mention in Lectures 15 and 25 that polynomial rootfinding is a highly unstable procedure for finding eigenvalues? The difference is that those remarks pertained to the idea of finding roots from the polynomial *coefficients*. Now, the idea is to find the roots by evaluating $p(x)$ at various points x, without ever looking at its coefficients, and applying the usual bisection process for nonlinear functions. This could be done, for example, by Gaussian elimination with pivoting (Exercise 21.1), and the resulting algorithm would be highly stable.

This much sounds useful enough, but not very exciting. What gives the bisection method its power and its appeal are some additional properties of eigenvalues and determinants that are not immediately obvious.

Given a symmetric matrix $A \in \mathbb{R}^{m \times m}$, let $A^{(1)}, \ldots, A^{(m)}$ denote its principal (i.e., upper-left) square submatrices of dimensions $1, \ldots, m$. It can be shown that the eigenvalues of these matrices *interlace*. Before defining this property, let us first sharpen it by assuming that A is tridiagonal and *irreducible* in the sense that all of its off-diagonal entries are nonzero:

$$A = \begin{bmatrix} a_1 & b_1 & & & \\ b_1 & a_2 & b_2 & & \\ & b_2 & a_3 & \ddots & \\ & & \ddots & \ddots & b_{m-1} \\ & & & b_{m-1} & a_m \end{bmatrix}, \qquad b_j \neq 0. \tag{30.5}$$

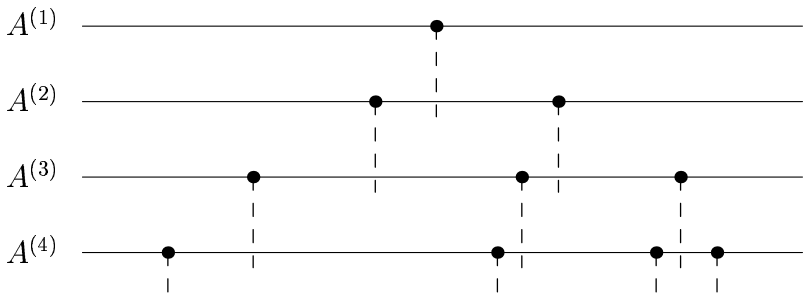


Figure 30.1. *Illustration of the strict eigenvalue interlace property* (30.6) *for the principal submatrices* $\{A^{(j)}\}$ *of an irreducible tridiagonal real symmetric matrix* A. *The eigenvalues of* $A^{(k)}$ *interlace those of* $A^{(k+1)}$. *The bisection algorithm takes advantage of this property.*

(If there are zeros on the off-diagonal, then the eigenvalue problem can be deflated, as in Algorithm 28.2.) By Exercise 25.1, the eigenvalues of $A^{(k)}$ are distinct; let them be denoted by $\lambda_1^{(k)} < \lambda_2^{(k)} < \cdots < \lambda_k^{(k)}$. The crucial property that makes bisection powerful is that these eigenvalues *strictly interlace*, satisfying the inequalities

$$\lambda_j^{(k+1)} < \lambda_j^{(k)} < \lambda_{j+1}^{(k+1)} \tag{30.6}$$

for $k = 1, 2, \ldots, m-1$ and $j = 1, 2, \ldots, k-1$. This behavior is sketched in Figure 30.1.

It is the interlacing property that makes it possible to count the exact number of eigenvalues of a matrix in a specified interval. For example, consider the 4×4 tridiagonal matrix

$$A = \begin{bmatrix} 1 & 1 & & \\ 1 & 0 & 1 & \\ & 1 & 2 & 1 \\ & & 1 & -1 \end{bmatrix}.$$

From the numbers

$$\det(A^{(1)}) = 1, \quad \det(A^{(2)}) = -1, \quad \det(A^{(3)}) = -3, \quad \det(A^{(4)}) = 4,$$

we know that $A^{(1)}$ has no negative eigenvalues, $A^{(2)}$ has one negative eigenvalue, $A^{(3)}$ has one negative eigenvalue, and $A^{(4)}$ has two negative eigenvalues. In general, for any symmetric tridiagonal $A \in \mathbb{R}^{m \times m}$, *the number of negative eigenvalues is equal to the number of sign changes in the sequence*

$$1, \ \det(A^{(1)}), \ \det(A^{(2)}), \ldots, \ \det(A^{(m)}), \tag{30.7}$$

which is known as a *Sturm sequence*. (This prescription works even if zero determinants are encountered along the way, if we define a "sign change" to

mean a transition from + or 0 to − or from − or 0 to + but not from + or − to 0.) By shifting A by a multiple of the identity, we can determine the number of eigenvalues in any interval $[a, b)$: it is the number of eigenvalues in $(-\infty, b)$ minus the number in $(-\infty, a)$.

One more observation completes the description of the bisection algorithm: for a tridiagonal matrix, the determinants of the matrices $\{A^{(k)}\}$ are related by a three-term recurrence relation. Expanding $\det(A^{(k)})$ by minors with respect to its entries b_{k-1} and a_k in row k gives, from (30.5),

$$\det(A^{(k)}) = a_k \det(A^{(k-1)}) - b_{k-1}^2 \det(A^{(k-2)}). \tag{30.8}$$

Introducing the shift by xI and writing $p^{(k)}(x) = \det(A^{(k)} - xI)$, we get

$$p^{(k)}(x) = (a_k - x)\, p^{(k-1)}(x) - b_{k-1}^2 p^{(k-2)}(x). \tag{30.9}$$

If we define $p^{(-1)}(x) = 0$ and $p^{(0)}(x) = 1$, then this recurrence is valid for all $k = 1, 2, \ldots, m$.

By applying (30.9) for a succession of values of x and counting sign changes along the way, the bisection algorithm locates eigenvalues in arbitrarily small intervals. The cost is $O(m)$ flops for each evaluation of the sequence, hence $O(m \log(\epsilon_{\text{machine}}))$ flops in total to find an eigenvalue to relative accuracy $\epsilon_{\text{machine}}$. If a small number of eigenvalues are needed, this is a distinct improvement over the $O(m^2)$ operation count for the QR algorithm. On a multiprocessor computer, multiple eigenvalues can be found independently on separate processors.

Divide-and-Conquer

The divide-and-conquer algorithm, based on a recursive subdivision of a symmetric tridiagonal eigenvalue problem into problems of smaller dimension, represents the most important advance in matrix eigenvalue algorithms since the 1960s. First introduced by Cuppen in 1981, this method is more than twice as fast as the QR algorithm if eigenvectors as well as eigenvalues are required.

We shall give just the essential idea, omitting all details. But the reader is warned that in this area, the details are particularly important, for the algorithm is not fully stable unless they are gotten right—a matter that was not well understood for a decade after Cuppen's original paper.

Let $T \in \mathbb{R}^{m \times m}$ with $m \geq 2$ be symmetric, tridiagonal, and irreducible in the sense of having only nonzeros on the off-diagonal. (Otherwise, the problem can be deflated.) Then for any n in the range $1 \leq n < m$, T can be split into

submatrices as follows:

$$T = \begin{bmatrix} T_1 & \beta \\ \beta & T_2 \end{bmatrix} = \begin{bmatrix} \hat{T}_1 & \\ & \hat{T}_2 \end{bmatrix} + \begin{bmatrix} & \\ \beta & \beta \\ \beta & \beta \end{bmatrix}. \tag{30.10}$$

Here T_1 is the upper-left $n \times n$ principal submatrix of T, T_2 is the lower-right $(m-n) \times (m-n)$ principal submatrix, and $\beta = t_{n+1,n} = t_{n,n+1} \neq 0$. The only difference between T_1 and $\hat{T}_1$ is that the lower-right entry t_{nn} has been replaced by $t_{nn} - \beta$, and the only difference between T_2 and $\hat{T}_2$ is that the upper-left entry $t_{n+1,n+1}$ has been replaced by $t_{n+1,n+1} - \beta$. These modifications of two entries are introduced to make the rightmost matrix of (30.10) have rank one.

Here is how (30.10) might be expressed in words. *A tridiagonal matrix can be written as the sum of a* 2×2 *block-diagonal matrix with tridiagonal blocks and a rank-one correction.*

The divide-and-conquer algorithm proceeds as follows. Split the matrix T as in (30.10) with $n \approx m/2$. Suppose the eigenvalues of $\hat{T}_1$ and $\hat{T}_2$ are known. Since the correction matrix is of rank one, a nonlinear but rapid calculation can be used to get from the eigenvalues of $\hat{T}_1$ and $\hat{T}_2$ to those of T itself. Now recurse on this idea, finding the eigenvalues of $\hat{T}_1$ and $\hat{T}_2$ by further subdivisions with rank-one corrections, and so on. In this manner an $m \times m$ eigenvalue problem is reduced to a set of 1×1 eigenvalue problems together with a collection of rank-one corrections. (In practice, for maximal efficiency, it is customary to switch to the QR algorithm when the submatrices are of sufficiently small dimension rather than to carry the recursion all the way.)

In this process there is one key mathematical point. If the eigenvalues of $\hat{T}_1$ and $\hat{T}_2$ are known, how can those of T be found? To answer this, suppose that diagonalizations

$$\hat{T}_1 = Q_1 D_1 Q_1^T, \qquad \hat{T}_2 = Q_2 D_2 Q_2^T$$

have been computed. Then from (30.10) it follows that we have

$$T = \begin{bmatrix} Q_1 & \\ & Q_2 \end{bmatrix} \left(\begin{bmatrix} D_1 & \\ & D_2 \end{bmatrix} + \beta z z^T \right) \begin{bmatrix} Q_1^T & \\ & Q_2^T \end{bmatrix} \tag{30.11}$$

with $z^T = (q_1^T, q_2^T)$, where q_1^T is the last row of Q_1 and q_2^T is the first row of Q_2. Since this equation is a similarity transformation, we have reduced the mathematical problem to the problem of finding the eigenvalues of a diagonal matrix plus a rank-one correction.

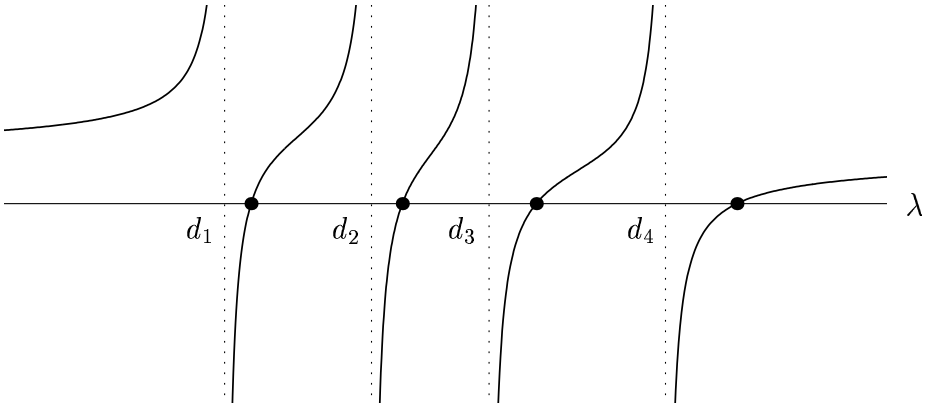


Figure 30.2. *Plot of the function $f(\lambda)$ of* (30.12) *for a problem of dimension* 4. *The poles of $f(\lambda)$ are the eigenvalues $\{d_j\}$ of D, and the roots of $f(\lambda)$ (solid dots) are the eigenvalues of $D + ww^T$. The rapid determination of these roots is the basis of each recursive step of the divide-and-conquer algorithm.*

To show how this is done, we simplify notation as follows. Suppose we wish to find the eigenvalues of $D + ww^T$, where $D \in \mathbb{R}^{m\times m}$ is a diagonal matrix with distinct diagonal entries $\{d_j\}$ and $w \in \mathbb{R}^m$ is a vector. (The choice of a plus sign corresponds to $\beta > 0$ above; for $\beta < 0$ we would consider $D - ww^T$.) We can assume $w_j \neq 0$ for all j, for otherwise, the problem is reducible. Then the eigenvalues of $D + ww^T$ are the roots of the rational function

$$f(\lambda) \;=\; 1 + \sum_{j=1}^{m} \frac{w_j^2}{d_j - \lambda}, \tag{30.12}$$

as illustrated in Figure 30.2. This assertion can be justified by noting that if $(D + ww^T)q = \lambda q$ for some $q \neq 0$, then $(D - \lambda I)q + w(w^T q) = 0$, implying $q + (D - \lambda I)^{-1}w(w^T q) = 0$, that is, $w^T q + w^T(D - \lambda I)^{-1}w(w^T q) = 0$. This amounts to the equation $f(\lambda)(w^T q) = 0$, in which $w^T q$ must be nonzero, for otherwise q would be an eigenvector of D, hence nonzero in only one position, implying $w^T q \neq 0$ after all. We conclude that if q is an eigenvector of $D + ww^T$ with eigenvalue λ, then $f(\lambda)$ must be 0, and the converse follows because the form of $f(\lambda)$ guarantees that it has exactly m zeros. The equation $f(\lambda) = 0$ is known as the *secular equation*.

At each recursive step of the divide-and-conquer algorithm, the roots of (30.12) are found by a rapid iterative process related to Newton's method. Only $O(1)$ iterations are required for each root (or $O(\log(|\log(\epsilon_{\text{machine}})|))$ iterations if $\epsilon_{\text{machine}}$ is viewed as a variable), making the operation count $O(m)$ flops per root for an $m \times m$ matrix, or $O(m^2)$ flops all together. If we imagine a recursion in which a matrix of dimension m is split exactly in half at each step, the total operation count for finding eigenvalues of a tridiagonal matrix

by the divide-and-conquer algorithm becomes

$$O\left(m^2 + 2\left(\frac{m}{2}\right)^2 + 4\left(\frac{m}{4}\right)^2 + 8\left(\frac{m}{8}\right)^2 + \cdots + m\left(\frac{m}{m}\right)^2\right), \qquad (30.13)$$

a series which converges to $O(m^2)$ (not $O(m^2 \log m)$) thanks to the squares in the denominators. Thus the operation count would appear to be of the same order $O(m^2)$ as for the QR algorithm.

So far, it is not clear why the divide-and-conquer algorithm is advantageous. Since the reduction of a full matrix to tridiagonal form ("Phase 1" in the terminology of Lecture 25) requires $4m^3/3$ flops (26.2), it would seem that any improvement in the $O(m^2)$ operation count for diagonalization of that tridiagonal matrix ("Phase 2") is hardly important. However, the economics change if one is computing eigenvectors as well as eigenvalues. Now, Phase 1 requires $8m^3/3$ flops but Phase 2 also requires $O(m^3)$ flops—for the QR algorithm, $\approx 6m^3$. The divide-and-conquer algorithm reduces this figure, ultimately because its nonlinear iterations involve just the scalar function (30.12), not the orthogonal matrices Q_j, whereas the QR algorithm must manipulate matrices Q_j at every iterative step.

An operation count reveals the following. The $O(m^3)$ part of the divide-and-conquer computation is the multiplication by Q_j and Q_j^T in (30.11). The total operation count, summed over all steps of the recursion, is $4m^3/3$ flops, a great improvement over $\approx 6m^3$ flops. Adding in the $8m^3/3$ flops for Phase 1 gives an improvement from $\approx 9m^3$ to $4m^3$.

Actually, the divide-and-conquer algorithm usually does even better than this, for a reason that is not elementary. For most matrices A, many of the vectors z and matrices Q_j that arise in (30.11) turn out to be numerically sparse in the sense that many of their entries have relative magnitudes less than machine precision. This sparsity allows a process of *numerical deflation*, whereby successive tridiagonal eigenvalue problems are reduced to uncoupled problems of smaller dimensions. In typical cases this reduces the Phase 2 operation count to an order less than m^3 flops, reducing the operation count for Phases 1 and 2 combined to $8m^3/3$. For eigenvalues alone, (30.13) becomes an overestimate and the Phase 2 operation count is reduced to an order lower than m^2 flops. The root of this fascinating phenomenon of deflation, which we shall not discuss further, is the fact that most of the eigenvectors of most tridiagonal matrices are "exponentially localized" (Exercise 30.7)—a fact that has been related by physicists to the phenomenon that glass is transparent.

We have spoken as if there is a single divide-and-conquer algorithm, but in fact, there are many variants. More complicated rank-one updates are often used for stability reasons, and rank-two updates are also sometimes used. Various methods are employed for finding the roots of $f(\lambda)$, and for large m, the fastest way to carry out the multiplications by Q_j is via multipole expansions rather than the obvious algorithm. A high-quality implementation of a divide-and-conquer algorithm can be found in the LAPACK library.

Exercises

30.1. Derive the formula (30.4), and give a precise geometric interpretation of the transformation (30.1) based on this choice of θ.

30.2. How many flops are required for one step (30.1) of the Jacobi algorithm? How many flops for $m(m-1)/2$ such steps, i.e., one sweep? How does the operation count for one sweep compare with the total operation count for tridiagonalizing a real symmetric matrix and finding its eigenvalues by the QR algorithm?

30.3. Show that if the largest off-diagonal entry is annihilated at each step of the Jacobi algorithm, then the sum of the squares of the off-diagonal entries decreases by at least the factor $1 - 2/(m^2 - m)$ at each step.

30.4. Suppose m is even and your computer has $m/2$ processors. Explain how $m/2$ transformations (30.1) can be carried out in parallel if they involve the disjoint row/column pairs $(1,2), (3,4), (5,6), \ldots, (m-1, m)$.

30.5. Write a program to find the eigenvalues of an $m \times m$ real symmetric matrix by the Jacobi algorithm with the standard row-wise ordering, plotting the sum of the squares of the off-diagonal entries on a log scale as a function of the number of sweeps. Apply your program to random matrices of dimensions 20, 40, and 80.

30.6. How many eigenvalues does

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}$$

have in the interval $[1, 2]$? Work out the answer on paper by bisection, making use of the recurrence (30.9).

30.7. Construct a random real symmetric tridiagonal matrix T of dimension 100 and compute its eigenvalue decomposition, $T = QDQ^T$. Plot a few of the eigenvectors on a log scale (the absolute values of a few columns of Q) and observe the phenomenon of localization. What proportion of the 10,000 entries of Q are greater than 10^{-10} in magnitude? What is the answer if instead of a random matrix, T is the discrete Laplacian with entries $1, -2, 1$?

Lecture 31. Computing the SVD

The computation of the SVD of an arbitrary matrix can be reduced to the computation of the eigenvalue decomposition of a hermitian square matrix, but the most obvious way of doing this is not stable. Instead, the standard methods for computing the SVD are based implicitly on another kind of reduction to hermitian form. For speed, the matrix is first unitarily bidiagonalized.

SVD of A and Eigenvalues of A^*A

As stated in Theorem 5.4, the SVD of the $m \times n$ matrix A ($m \geq n$), $A = U\Sigma V^*$, is related to the eigenvalue decomposition of the matrix A^*A,

$$A^*A = V\Sigma^*\Sigma V^*.$$

Thus, mathematically speaking, we might calculate the SVD of A as follows:

1. Form A^*A;
2. Compute the eigenvalue decomposition $A^*A = V\Lambda V^*$;
3. Let Σ be the $m \times n$ nonnegative diagonal square root of Λ;
4. Solve the system $U\Sigma = AV$ for unitary U (e.g., via QR factorization).

This algorithm is frequently used, often by people who have rediscovered the SVD for themselves. The matrix A^*A is known as the *covariance matrix* of A, and it has familiar interpretations in statistics and other fields. The algorithm

is unstable, however, because it reduces the SVD problem to an eigenvalue problem that may be much more sensitive to perturbations.

The difficulty can be explained as follows. We have seen that when a hermitian matrix A^*A is perturbed by δB, the absolute changes in each eigenvalue are bounded by the 2-norm of the perturbation. By Exercise 26.3(b), $|\lambda_k(A^*A+\delta B)-\lambda_k(A^*A)| \le \|\delta B\|_2$. As is implied by equation (31.2) below, a similar bound holds for the singular values of A itself, $|\sigma_k(A+\delta A)-\sigma_k(A)| \le \|\delta A\|_2$. Thus a backward stable algorithm for computing singular values would obtain $\tilde{\sigma}_k$ satisfying

$$\tilde{\sigma}_k = \sigma_k(A+\delta A), \qquad \frac{\|\delta A\|}{\|A\|} = O(\epsilon_{\text{machine}}), \tag{31.1}$$

which would imply

$$|\tilde{\sigma}_k - \sigma_k| = O(\epsilon_{\text{machine}}\|A\|).$$

Now observe what happens if we proceed by computing $\lambda_k(A^*A)$. If $\lambda_k(A^*A)$ is computed stably, we must expect errors of the order

$$|\tilde{\lambda}_k - \lambda_k| \;=\; O(\epsilon_{\text{machine}}\|A^*A\|) \;=\; O(\epsilon_{\text{machine}}\|A\|^2).$$

Square-rooting to get σ_k, we find

$$|\tilde{\sigma}_k - \sigma_k| \;=\; O(|\tilde{\lambda}_k - \lambda_k|/\sqrt{\lambda_k}) \;=\; O(\epsilon_{\text{machine}}\|A\|^2/\sigma_k).$$

This is worse than the previous result by a factor $O(\|A\|/\sigma_k)$. This is no problem for the dominant singular values of A, with $\sigma_k \approx \|A\|$, but it is a big problem for any singular values with $\sigma_k \ll \|A\|$. For the smallest singular value σ_n, we must expect a loss of accuracy of order $\kappa(A)$—a "squaring of the condition number," just as in the use of the normal equations for certain least squares problems (Lecture 19).

A Different Reduction to an Eigenvalue Problem

There is an alternative, stable way to reduce the SVD to an eigenvalue problem. Assume that A is square, with $m = n$; this is no essential restriction, since we shall see that rectangular singular value problems can be reduced to square ones. Consider the $2m \times 2m$ hermitian matrix

$$H \;=\; \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix} \tag{31.2}$$

mentioned earlier in Exercise 5.4. Since $A = U\Sigma V^*$ implies $AV = U\Sigma$ and $A^*U = V\Sigma^* = V\Sigma$, we have the block 2×2 equation

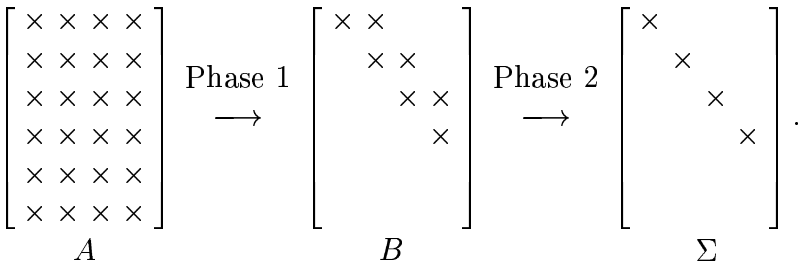
$$\begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix} \begin{bmatrix} V & V \\ U & -U \end{bmatrix} \;=\; \begin{bmatrix} V & V \\ U & -U \end{bmatrix} \begin{bmatrix} \Sigma & 0 \\ 0 & -\Sigma \end{bmatrix}, \tag{31.3}$$

which amounts to an eigenvalue decomposition of H. Thus we see that the singular values of A are the absolute values of the eigenvalues of H, and the singular vectors of A can be extracted from the eigenvectors of H.

Thus one could obtain the SVD of a square matrix A by forming the matrix H and computing its eigenvalue decomposition. In contrast to the use of AA^* or A^*A, this approach is stable. The standard algorithms for the SVD are based on this idea, though in a disguised manner in which no matrices of dimension as large as $m+n$ are formed explicitly. And the key step to make the process fast is an initial unitary reduction to bidiagonal form.

Two Phases

We have seen that hermitian eigenvalue problems are usually solved by a two-phase computation: first reduce the matrix to tridiagonal form, then diagonalize the tridiagonal matrix. Since the work of Golub, Kahan, and others in the 1960s, an analogous two-phase approach has been standard for the SVD. The matrix is brought into bidiagonal form, and then the bidiagonal matrix is diagonalized:



Phase 1 involves a finite number of operations, $O(mn^2)$ flops. Phase 2 in principle requires an infinite number of operations, but the standard algorithms converge superlinearly, and thus only $O(n \log(|\log(\epsilon_{\text{machine}})|))$ iterations are needed for convergence to order $\epsilon_{\text{machine}}$ (Exercise 25.2). In practice, we think of $\epsilon_{\text{machine}}$ as a constant and say that convergence is achieved in $O(n)$ iterations. Because the matrix operated on is bidiagonal, each of these iterations requires only $O(n)$ flops. Phase 2 therefore requires $O(n^2)$ flops all together (assuming singular values but not vectors are required). Thus, although Phase 1 is finite and Phase 2 is in principle infinite, in practice the latter is much the less expensive, just as we found for the symmetric eigenvalue problem.

Golub–Kahan Bidiagonalization

In Phase 1 of the SVD computation, we bring A into bidiagonal form by applying distinct unitary operations on the left and right. Note how this differs from the computation of eigenvalues, where the same unitary operations must be applied on both sides so that each step is a similarity transformation. In

that case, it was only possible to introduce zeros below the first subdiagonal. Here, we are able to completely triangularize and also introduce zeros above the first superdiagonal.

The simplest method for accomplishing this, *Golub–Kahan bidiagonalization*, proceeds as follows. Householder reflectors are applied alternately on the left and the right. Each left reflection introduces a column of zeros below the diagonal. The right reflection that follows introduces a row of zeros to the right of the first superdiagonal, leaving intact the zeros just introduced in the column. For example, for a 6×4 matrix, the first two pairs of reflections look like this:

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}} \xrightarrow{U_1^* \cdot} \underset{U_1^* A}{\begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}} \xrightarrow{\cdot V_1} \underset{U_1^* A V_1}{\begin{bmatrix} \times & \times & 0 & 0 \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \\ & \times & \times & \times \end{bmatrix}}$$

$$\xrightarrow{U_2^* \cdot} \underset{U_2^* U_1^* A V_1}{\begin{bmatrix} \times & \times & & \\ & \times & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \\ & 0 & \times & \times \end{bmatrix}} \xrightarrow{\cdot V_2} \underset{U_2^* U_1^* A V_1 V_2}{\begin{bmatrix} \times & \times & & \\ & \times & \times & 0 \\ & & \times & \times \\ & & \times & \times \\ & & \times & \times \\ & & \times & \times \end{bmatrix}}.$$

The left-multiplication by U_1^* modifies rows 1 to m, introducing zeros in column 1 below the diagonal. The right-multiplication by V_1 modifies columns 2 to n, introducing zeros in row 1 without destroying the zeros in column 1. The process continues with operations on rows 2 to m, then columns 3 to n, and so on.

At the end of this process, n reflectors have been applied on the left and $n-2$ on the right. The pattern of floating point operations resembles two Householder QR factorizations interleaved with each other, one operating on the $m \times n$ matrix A, the other on the $n \times m$ matrix A^*. The total operation count is therefore twice that of a QR factorization (10.9), i.e.,

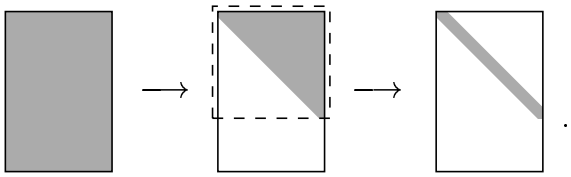
$$\text{Work for Golub–Kahan bidiagonalization:} \quad \sim 4mn^2 - \frac{4}{3}n^3 \text{ flops.} \qquad (31.4)$$

Faster Methods for Phase 1

For $m \gg n$, this operation count is unnecessarily large. A single QR factorization would introduce zeros everywhere below the diagonal, and for $m \gg n$,

these are the great majority of the zeros that are needed. Yet the operation count for the Golub–Kahan method is twice as high. This observation suggests an alternative method for bidiagonalization with $m \gg n$, first proposed by Lawson and Hanson and later developed by Chan. The idea, *LHC bidiagonalization*, is illustrated as follows:

Lawson–Hanson–Chan bidiagonalization



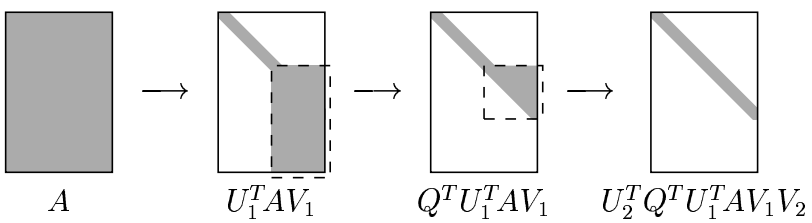
We begin by computing the QR factorization $A = QR$. Then we compute the Golub–Kahan bidiagonalization $B = U^*RV$ of R. The QR factorization requires $2mn^2 - \frac{2}{3}n^3$ flops (10.9), and the Golub–Kahan procedure, which now only has to operate on the upper $n \times n$ submatrix, requires $\frac{8}{3}n^3$ flops. The total operation count is

$$\text{Work for LHC bidiagonalization:} \quad \sim 2mn^2 + 2n^3 \text{ flops.} \tag{31.5}$$

This is cheaper than Golub–Kahan bidiagonalization for $m > \frac{5}{3}n$ (Exercise 31.1). Curiously, the LHC procedure creates zeros and then destroys them again (in the lower triangle of the upper $n \times n$ square of A), but there is a net gain.

The LHC procedure is advantageous only when $m > \frac{5}{3}n$, but the idea can be generalized so as to realize a saving for any $m > n$. The trick is to apply the QR factorization not at the beginning of the computation, but at a suitable point in the middle. This is advantageous because in the Golub–Kahan process, a matrix with $m > n$ becomes skinnier as the bidiagonalization proceeds. If the initial aspect ratio is, say, $m/n = 3/2$, it will steadily grow to $5/3$ and 2 and beyond. After step k, the aspect ratio of the remaining matrix is $(m-k)/(n-k)$, and when this figure gets sufficiently large, it makes sense to perform a QR factorization to reduce the problem to a square matrix.

Three-step bidiagonalization



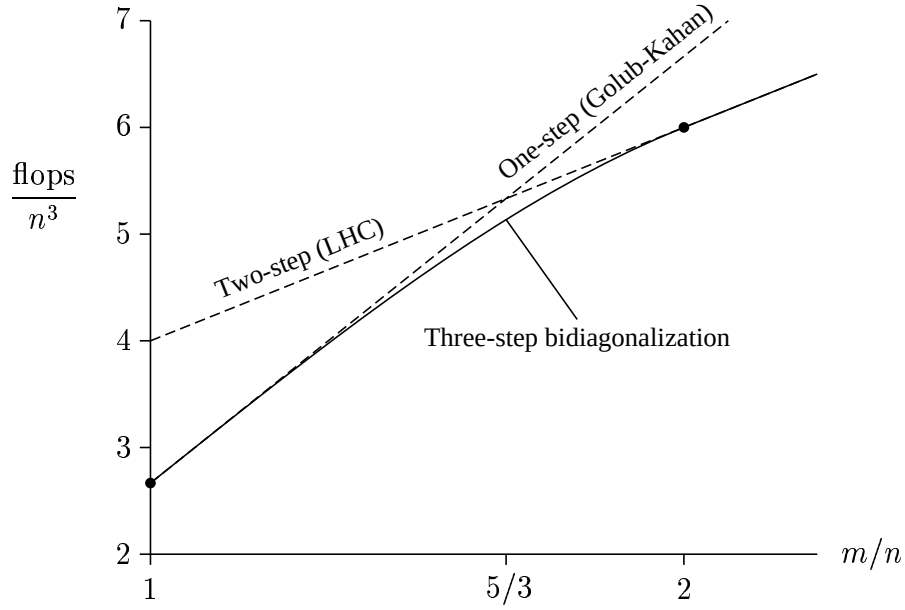


Figure 31.1. *Operation counts for three bidiagonalization algorithms applied to $m \times n$ matrices, from* (31.4), (31.5), *and* (31.6). *Three-step bidiagonalization provides a pleasingly smooth interpolant between the other two methods, though the improvement is hardly large.*

When should the QR factorization be performed? If we aim solely to minimize the operation count, the answer is simple: when the aspect ratio reaches $(m-k)/(n-k) = 2$ (Exercise 31.2). This choice leads to the formula

$$\text{Work for three-step bidiagonalization:} \quad \sim 4mn^2 - \frac{4}{3}n^3 - \frac{2}{3}(m-n)^3 \text{ flops}, \tag{31.6}$$

a modest improvement over the other two methods for $n < m < 2n$.

The operation counts for the three methods are plotted as a function of m/n in Figure 31.1. It must be admitted that the improvement achieved by the three-step method is small enough that in practice, other matters besides the count may determine which method is best on a real machine (see p. 59).

Phase 2

In Phase 2 of the computation of the SVD, the SVD of the bidiagonal matrix B is determined. From the 1960s to the 1990s, the standard algorithm for this was a variant of the QR algorithm. More recently, divide-and-conquer algorithms have also become competitive, and in the future, they are likely to become the standard. We shall not give details.

Exercises

31.1. (a) Show that, as claimed in the text and illustrated in Figure 31.1, the crossover aspect ratio at which LHC bidiagonalization begins to beat Golub–Kahan bidiagonalization is $m/n = 5/3$.

(b) By what fraction is three-step bidiagonalization faster than the other two methods for $m/n = 5/3$?

31.2. Show that in three-step bidiagonalization, the optimal point at which to perform the QR factorization is when the matrix reaches an aspect ratio of 2.

31.3. Show that if the entries on both principal diagonals of a bidiagonal matrix are all nonzero, then the singular values of the matrix are distinct. (See Exercise 25.1.)

31.4. Let A be the $m \times m$ upper-triangular matrix with 0.1 on the main diagonal and 1 everywhere above the diagonal. Write a program to compute the smallest singular value of A in two ways: by calling a standard SVD software, and by forming A^*A and computing the square root of its smallest eigenvalue. Run your program for $1 \le m \le 30$ and plot the results as two curves on a log scale. Do the results conform to our general discussion of these algorithms?

31.5. Let A be an $m \times n$ matrix whose entries are independent samples from $N(0,1)$, the normal distribution of mean 0, variance 1 (compare Exercise 12.3). Let B be a bidiagonal matrix

$$B = \begin{bmatrix} x_m & y_{n-1} & & & & \\ & x_{m-1} & y_{n-2} & & & \\ & & \ddots & \ddots & & \\ & & & x_{m-(n-2)} & y_1 & \\ & & & & x_{m-(n-1)} & \end{bmatrix},$$

where each x or y is the positive square root of an independent sample from the χ^2 distribution with degree equal to the attached subscript. (The χ^2 distribution of degree k is equal to the distribution of the sum of squares of k independent variables from $N(0,1)$.)

(a) Show that the distributions of the singular values of A and B are the same.

(b) Verify this result by an experiment. Specifically, take $m = 100$ and $n = 50$, construct random matrices A and B as indicated, and plot the singular values of A against those of B.